claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_cb1d52fa-fed\agent-a4ff07840267bb18b.jsonl`
- SHA-256: `cd7109162f0f0b58ee9971508c72d12072676c3807f3f39c1a5a0b0ace9d5e95`
- Source modified: `2026-06-14T08:27:11+00:00`
- Imported at: `2026-07-05T16:48:25+00:00`
- project: `wf_cb1d52fa-fed`
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`

Transcript

1. user (2026-06-14T08:24:06.516Z)

THE PIVOT under study = "PERSONALIZATION-FIRST": instead of one universal model for all users, ship a BATTERIES-INCLUDED baseline that CONTINUALLY SELF-OPTIMIZES to a SINGLE user's own footage. As the user's corpus grows (10-15 videos), the on-device model + PER-USER CUE SELECTION (driven by the EXISTING LOVO / A-B experiment harness run on the user's OWN videos, with the user's CORRECTION LOOP as the per-user label source) becomes heavily tuned for THAT user ? deep analysis of THEIR rallies + shot types (serves, smashes, drives, drop shots). Per-user decisions like "the motion/person cue gives no F1 win for THIS user -> default it OFF for them." The model is optimized for the user's videos, NOT others (wins may hold anecdotally on other footage but weaker). Periodic "BATTERIES UPGRADES" improve the shipped baseline so it needs merely <5 videos to reach superior quality. Two user archetypes: (a) an economically-generous hobbyist who records sessions some days; (b) a professional with 100+ hours of their own video. CONTRAST = "generalization-first" (build one model that works on everyone's videos). The owner believes the harness + eval + A/B were DESIGNED to enable this roadmap.

DIMENSION: HOW THE EXISTING HARNESS/ARCHITECTURE SUPPORTS IT (verify the owner's claim that it was designed for this). Read backend/eval/experiment.py (Variant, compare, LOVO, promote-only-on-lift, record_experiment leaderboard), backend/eval/eval_stats.py (C1-C4 honest small-N stats), backend/eval/fusion_golden.py (replayable per-video feature substrate), backend/pipeline/detectors/fusion_features.py (signals-as-feature-columns, default-OFF). Map each to PER-USER optimization: a per-user corpus is just a special case of the corpus the harness already A/B's; "motion cue off for this user" IS a per-user Variant decision; the replayable substrate = re-score a user's videos as their corpus grows; promote-only-on-lift = per-user cue gating. State concretely what is ALREADY built for this vs what's MISSING (e.g. per-user model store/versioning, the correction-loop -> per-user-labels pipeline, on-device training loop, per-user eval reporting). Return pros/cons + the gap list.

2. assistant (2026-06-14T08:24:09.746Z)

I'll analyze how the existing harness/architecture supports the personalization-first pivot. Let me start by reading the key files and project context.

3. user (2026-06-14T08:24:12.772Z)

Already up to date.

4. user (2026-06-14T08:24:12.781Z)

```
1       """Honest small-N evaluation statistics (course-corrections C1 + C3).
2
3       The ~6-match golden corpus + a regularised logistic arbiter is exactly the regime where
a
4       single leave-one-video-out *mean* lies: leave-one-out anti-correlates each fold's
training
5       and held-out label means at small N, and one lucky/unlucky fold swings the headline. The
```

```
6       ``compare``/LOVO machinery already produces **per-video held-out scores** ? this module
7       turns those into an honest summary:
8
9       - **C1 ? never a single mean.** ``mean_with_ci`` / ``bootstrap_ci`` report a confidence
10        interval; ``paired_sign_test`` and ``paired_delta_ci`` grade a B?A improvement by how
11        many *videos* it wins on (the distribution-free test the repo already leans on:
12        "sign test needs n ? ~n=10/9-wins for p<0.05"), so the promotion gate can't promote
noise.
13      - **C3 ? honest stacking.** ``out_of_fold_predictions`` / ``grouped_folds`` give a
learned
14        producer's *out-of-fold* outputs so feeding them to the arbiter doesn't leak (group by
15        match, never a held-out window from the same video).
16
17      See ``docs/ANALYZERS_AND_RECONCILERS.md`` §13/C1+C3. All **additive**, pure (stdlib
only),
18      deterministic (bootstrap takes an explicit ``seed``) ? nothing here changes existing
19      ``compare``/``calibration.leave_one_video_out`` behaviour; callers opt in.
20      """
21      from __future__ import annotations
22
23      import math
24      import random
25      import statistics
26      from typing import Any, Callable, Dict, List, Optional, Sequence, Tuple
27
28      Interval = Tuple[float, float]
29      FitPredict = Callable[[List[int], List[int]], List[Any]]
30
31
32      def bootstrap_ci(values: Sequence[float], confidence: float = 0.95,
33                       n_boot: int = 2000, seed: int = 0) -> Tuple[float, float]:
34          """Percentile bootstrap CI for the mean of ``values``. Deterministic given ``seed``.
35
36          ``n<=1`` returns a degenerate ``(v, v)`` / ``(0, 0)`` interval ? there is no spread
to
37          resample. With n=6 (our corpus) this is the honest "how wide could the mean be?"
band.
38          """
39          vals = [float(v) for v in values]
40          n = len(vals)
41          if n == 0:
42              return (0.0, 0.0)
43          if n == 1:
44              return (vals[0], vals[0])
45          rng = random.Random(seed)
46          means: List[float] = []
47          for _ in range(max(1, n_boot)):
48              means.append(sum(vals[rng.randrange(n)] for _ in range(n)) / n)
49          means.sort()
50          alpha = (1.0 - confidence) / 2.0
51          last = len(means) - 1
52          lo = means[max(0, int(math.floor(alpha * last)))]
53          hi = means[min(last, int(math.ceil((1.0 - alpha) * last)))]
54          return (lo, hi)
55
56
57      def mean_with_ci(values: Sequence[float], confidence: float = 0.95,
58                       n_boot: int = 2000, seed: int = 0) -> Dict[str, float]:
59          """``{mean, std, n, ci_low, ci_high, confidence}`` ? report this, never a bare mean
(C1)."""
60          vals = [float(v) for v in values]
61          n = len(vals)
62          mean = statistics.mean(vals) if vals else 0.0
63          std = statistics.pstdev(vals) if n > 1 else 0.0
64          lo, hi = bootstrap_ci(vals, confidence, n_boot, seed)
```

```
        return {"mean": mean, "std": std, "n": float(n),
                "ci_low": lo, "ci_high": hi, "confidence": confidence}


    def paired_sign_test(deltas: Sequence[float], eps: float = 0.0) -> Dict[str, float]:
        """Two-sided exact sign test over per-fold deltas (B ? A).

        ``wins`` = folds where B beat A by more than ``eps``; ``losses`` the reverse; ties
        excluded. ``p_value`` is the exact two-sided binomial tail under p=0.5 ? the
        distribution-free "did B win on enough *videos*?" check.
        """
        wins = sum(1 for d in deltas if d > eps)
        losses = sum(1 for d in deltas if d < -eps)
        ties = len(deltas) - wins - losses
        n_eff = wins + losses
        if n_eff == 0:
            p = 1.0
        else:
            k = min(wins, losses)
            tail = sum(math.comb(n_eff, j) for j in range(k + 1)) * (0.5 ** n_eff)
            p = min(1.0, 2.0 * tail)
        return {"wins": float(wins), "losses": float(losses), "ties": float(ties),
                "n_effective": float(n_eff), "p_value": p}


    def paired_delta_ci(a_values: Sequence[float], b_values: Sequence[float],
                        confidence: float = 0.95, n_boot: int = 2000, seed: int = 0,
                        eps: float = 0.0) -> Dict[str, Any]:
        """Paired (by fold) B ? A summary: mean delta + bootstrap CI + the sign test (C1).

        ``a_values``/``b_values`` are per-fold held-out scores for variants A and B, aligned
by
        fold (same video order). A promotion is honest when the delta CI clears 0 *and* the
sign
        test agrees ? not when a single mean ticked up.
        """
        n = min(len(a_values), len(b_values))
        deltas = [float(b_values[i]) - float(a_values[i]) for i in range(n)]
        summary = mean_with_ci(deltas, confidence, n_boot, seed)
        return {
            "mean_delta": summary["mean"],
            "ci_low": summary["ci_low"],
            "ci_high": summary["ci_high"],
            "n": float(n),
            "ci_excludes_zero": (summary["ci_low"] > 0.0) or (summary["ci_high"] < 0.0),
            "sign_test": paired_sign_test(deltas, eps),
        }


    def grouped_folds(groups: Sequence[Any]) -> List[Tuple[List[int], List[int]]]:
        """Leave-one-group-out splits ``(train_idx, test_idx)`` ? group by match, the honest
unit.

        The field's name for "LOVO by match" is leave-one-group-out (LOGO). Group order is
        first-seen (deterministic). Groups with no complement (a single group) yield no
fold.
        """
        seen: List[Any] = []
        for g in groups:
            if g not in seen:
                seen.append(g)
        n = len(groups)
        folds: List[Tuple[List[int], List[int]]] = []
        for g in seen:
            test = [i for i in range(n) if groups[i] == g]
```

```
126             train = [i for i in range(n) if groups[i] != g]
127             if test and train:
128                 folds.append((train, test))
129         return folds
130
131
132     def out_of_fold_predictions(groups: Sequence[Any], fit_predict: FitPredict) ->
List[Optional[Any]]:
133         """Out-of-fold predictions for honest stacking (C3).
134
135         For each leave-one-group-out fold, ``fit_predict(train_idx, test_idx)`` trains on
the
136         other groups and returns predictions for ``test_idx`` (in that order); results are
137         scattered back to a list aligned to the input items. So a *learned* producer's
outputs,
138         when used as arbiter features, never include in-sample predictions ? the leakage
path
139         that inflates measured lift on a tiny corpus. Items in a singleton group stay
``None``
140         (needs >= 2 groups to predict).
141         """
142         n = len(groups)
143         oof: List[Optional[Any]] = [None] * n
144         for train_idx, test_idx in grouped_folds(groups):
145             preds = fit_predict(train_idx, test_idx)
146             for pos, ti in enumerate(test_idx):
147                 if pos < len(preds):
148                     oof[ti] = preds[pos]
149         return oof
150
```

5. user (2026-06-14T08:24:13.453Z)

```
1       """Experiment harness ? A/B + shadow + side-by-side for rally detection.
2
3       This is the thin **variant/experiment layer** that `docs/EXPERIMENT_HARNESS.md`
4       identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
5       no-regression gate, the pinnable classifier artifact, the persisted-candidate
6       substrate) already exists ? this module composes it. **No new scoring math lives
7       here**; everything defers to:
8
9         - `backend.eval.metrics`        ? `score`, `match_intervals`, `interval_iou`
10        - `backend.eval.training`       ? `label_candidates` (y by the same IoU matcher)
11        - `backend.eval.classifier`     ? `LogisticRegression` (the pinnable JSON model)
12        - `backend.eval.gemini_refine`  ? `merge_overlaps` (the blessed window merge)
13        - `backend.eval.regression`     ? `gt_hash` (label-set fingerprint)
14        - `backend.eval.calibration`    ? `pct_improvement`
15
16      The thesis (`EXPERIMENT_HARNESS.md` §2): separate the **expensive, risky DETECTION**
17      (per-frame signals ? run once on the GPU/WSL box, persisted into
18      `candidate_segments.stats`) from the **cheap, swappable DECISION** (given those
19      signals, where are the rallies). A `Variant` is a declarative re-scoring recipe;
20      given persisted candidate features it produces `List[Interval]` deterministically,
21      with no GPU and zero production risk. So most experiments are pure offline
22      re-scoring of stored data and never touch the served pipeline.
23
24      Three modes (`EXPERIMENT_HARNESS.md` §5):
25        - `compare()`           ? labeled A/B vs golden labels: per-video + LOVO-honest
26          aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
27          held-out loop, but variant-parameterised.
28        - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
29          variants agree / diverge (A-only, B-only, agreed). The divergences are what a
30          human spot-checks (and what two highlight reels would show side by side).
31        - the promotion gate stays `run_regression.py` + `regression_baseline.json`
32          (exit 0/1/3); **rollback = flip the variant/config, never `git revert`.**
```

```
33
34      Pure + offline + unit-tested. The only DB-touching helper is
35      `load_video_candidates`, which reads persisted candidates via
36      `Database.query_candidate_segments`.
37      """
38      from __future__ import annotations
39
40      import json
41      import logging
42      import os
43      from dataclasses import dataclass, field
44      from datetime import datetime, timezone
45      from hashlib import sha1
46      from typing import Any, Callable, Dict, List, Optional, Sequence
47
48      from backend.eval.calibration import pct_improvement
49      from backend.eval.classifier import LogisticRegression
50      from backend.eval.gemini_refine import merge_overlaps
51      from backend.eval.metrics import Interval, match_intervals, score
52      from backend.eval.regression import gt_hash
53      from backend.eval.training import label_candidates
54      from backend.eval import eval_stats as _stats           # C1: CIs + paired sign test
55      from backend.eval import segmentation_metrics as _segm   # C4: F1@k + segment-count
ratio
56      from backend.eval.score_calibration import fit_isotonic, fit_platt   # C2: calibrate cue
scores
57
58      logger = logging.getLogger(__name__)
59
60      # Where a comparison run appends one reproducible record. Tracked location (NOT under
61      # the gitignored output/) so the leaderboard survives a clean checkout, mirroring
62      # regression.DEFAULT_BASELINE_PATH.
63      DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")
64      _METRICS = ("precision", "recall", "f1", "mean_iou")
65
66
67      class ExperimentError(ValueError):
68          """A variant cannot be evaluated as configured (e.g. a learned variant asked to
69          score an unlabeled video with no pinned model, or a gate referencing an absent
70          feature)."""
71
72
73      # ---------------------------------------------------------------------- Variant
74
75
76      @dataclass
77      class Variant:
78          """A declarative re-scoring recipe ? NOT a code branch (`EXPERIMENT_HARNESS.md` §4).
79
80          A variant turns one video's persisted candidate windows + features into rally
81          intervals. Every knob defaults to the **control** behaviour (no gate, no learned
82          filter), so a variant that merely *carries* a new, disabled cue is byte-identical
83          to control ? the core no-regression guarantee.
84
85          Fields:
86            - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
87              for the leaderboard and for selecting the served path (the existing
88              `segmenter_registry` + `IndexingConfig` do the actual selection in production).
89              New cues are recorded here default-OFF.
90            - ``min_crossings`` ? decision-gate **Gate A**: keep only windows whose persisted
91              ``net_crossings`` feature is >= this. 0 = off. This is the eval-only gate from
92              `rally_gate.py` expressed as a re-scoring knob (kills service-feed / held-
shuttle
93              windows ? see `MULTI_SIGNAL_FUSION_PLAN.md` §3.1).
94            - ``min_players`` ? decision-gate **Gate B** (the future person cue): keep windows
```

```
95              whose persisted ``players_in_court`` is >= this. 0 = off (no-op until the person
96              cue persists that feature).
97            - ``classifier_model`` ? path to a frozen `LogisticRegression` JSON. When set,
98              `compare()` scores videos with the SHIPPED model as-is (no per-fold training);
99              required for `compare_unlabeled` on a learned variant.
100           - ``feature_names`` ? the persisted features the learned filter consumes. When set
101             WITHOUT ``classifier_model``, `compare()` trains a fresh model per LOVO fold
102             (honest) ? the recipe, not a pinned artifact.
103           - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
104             overlap-merge gap (seconds), the same `gemini_refine.merge_overlaps` default.
105         """
106
107         name: str
108         segmenter: str = "wasb_hybrid"
109         config_overrides: Dict[str, Any] = field(default_factory=dict)
110         cue_flags: Dict[str, bool] = field(default_factory=dict)
111         min_crossings: int = 0
112         min_players: int = 0
113         classifier_model: Optional[str] = None
114         feature_names: Optional[List[str]] = None
115         threshold: float = 0.5
116         merge_gap: float = 1.0
117         # C2 / threshold-in-LOVO (default OFF ? unchanged behaviour). When set, the
operating
118         # point is chosen on the TRAINING fold, never the held-out one ? fixing the
first-A/B
119         # failure where a fixed 0.5 zeroed out a small-candidate video.
120         tune_threshold: bool = False          # pick the F1-best threshold on the train fold
121         calibrate: Optional[str] = None       # None | "platt" | "isotonic" ? calibrate
proba first
122
123         def is_learned(self) -> bool:
124             """True if this variant applies a learned classifier (pinned model or
recipe)."""
125             return self.classifier_model is not None or bool(self.feature_names)
126
127         def to_dict(self) -> dict:
128             return {
129                 "name": self.name,
130                 "segmenter": self.segmenter,
131                 "config_overrides": self.config_overrides,
132                 "cue_flags": self.cue_flags,
133                 "min_crossings": self.min_crossings,
134                 "min_players": self.min_players,
135                 "classifier_model": self.classifier_model,
136                 "feature_names": self.feature_names,
137                 "threshold": self.threshold,
138                 "merge_gap": self.merge_gap,
139                 "tune_threshold": self.tune_threshold,
140                 "calibrate": self.calibrate,
141             }
142
143         @classmethod
144         def from_dict(cls, d: dict) -> "Variant":
145             known = {f for f in cls.__dataclass_fields__}  # type: ignore[attr-defined]
146             return cls(**{k: v for k, v in d.items() if k in known})
147
148         def spec_hash(self) -> str:
149             """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
150             and makes a result reproducible. The pinned **control** variant always hashes
the
151             same, so a regression is always traceable to a recipe change, never to drift."""
152             blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
153             return sha1(blob.encode()).hexdigest()[:12]
154
```

```

    #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
    #: filter. Always the comparison reference; "rollback" = make this the served variant.
    CONTROL = Variant(name="control")


    # ------------------------------------------------------------------ persisted candidates


    @dataclass
    class VideoCandidates:
        """One video's persisted detection, ready for offline re-scoring.

        ``intervals`` are the candidate windows; ``features`` is column-major
        (``feature_name -> per-candidate value``, each list aligned to ``intervals``);
        ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB
        with `load_video_candidates`, or directly in tests.
        """

        name: str
        intervals: List[Interval]
        features: Dict[str, List[float]] = field(default_factory=dict)
        gts: Optional[List[Interval]] = None


    def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
        """Persisted feature column, defaulting missing/short columns to 0.0 (matches
        `training.extract_yolo_features`, which lets a sparse/old row still vectorise)."""
        col = vc.features.get(name)
        n = len(vc.intervals)
        if col is None:
            logger.warning("variant references feature %r absent from %s ? treating as 0.0",
                           name, vc.name)
            return [0.0] * n
        if len(col) != n:
            raise ExperimentError(
                f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates")
        return [float(x) for x in col]


    def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) ->
List[List[float]]:
        cols = [_feature_column(vc, name) for name in feature_names]
        return [list(row) for row in zip(*cols)] if cols else [[] for _ in vc.intervals]


    def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
        """Boolean keep-mask from the decision gates (Gate A net-crossings, Gate B
players)."""
        n = len(vc.intervals)
        mask = [True] * n
        if variant.min_crossings > 0:
            col = _feature_column(vc, "net_crossings")
            mask = [m and (col[i] >= variant.min_crossings) for i, m in enumerate(mask)]
        if variant.min_players > 0:
            col = _feature_column(vc, "players_in_court")
            mask = [m and (col[i] >= variant.min_players) for i, m in enumerate(mask)]
        return mask


    def predict(variant: Variant, vc: VideoCandidates,
                model: Optional[LogisticRegression] = None,
                threshold: Optional[float] = None,
                calibrator: Optional[Callable[[float], float]] = None) -> List[Interval]:
        """Variant prediction: gate by persisted features, optionally filter by a learned
```

```
        model, then merge. Pure and deterministic.

        ``model`` (an already-fitted `LogisticRegression`) is supplied by `compare` per LOVO
        fold; if omitted and the variant pins ``classifier_model``, that frozen model is
        loaded. A learned variant with neither a supplied nor a pinned model raises ? there
        is nothing to score with. ``threshold``/``calibrator`` (both fitted on the TRAINING
        fold) override the variant defaults when the C2 / threshold-in-LOVO path supplies
them.
        """
        mask = _gate_mask(variant, vc)
        if variant.feature_names:
            if model is None:
                if variant.classifier_model is None:
                    raise ExperimentError(
                        f"variant {variant.name!r} is learned but has no model to predict "
                        f"with (pass a fitted model or set classifier_model)")
                model = LogisticRegression.load(variant.classifier_model)
            proba = [float(p) for p in model.predict_proba(_feature_matrix(vc,
variant.feature_names))]
            if calibrator is not None:
                proba = [calibrator(p) for p in proba]
            thr = variant.threshold if threshold is None else threshold
            mask = [m and (proba[i] >= thr) for i, m in enumerate(mask)]
        kept = [iv for iv, keep in zip(vc.intervals, mask) if keep]
        return merge_overlaps(kept, gap_tol=variant.merge_gap)


    def _calibrate_and_tune(variant: Variant, model: LogisticRegression,
                            train_videos: Sequence[VideoCandidates], iou: float
                            ) -> "tuple[Optional[Callable[[float], float]],
Optional[float]]":
        """Fit a calibrator and/or pick the operating threshold on the TRAINING fold only
        (C2 + threshold-in-LOVO). Returns ``(calibrator, threshold)`` ? either may be None
        (use the variant default). Honest: never looks at the held-out video.
        """
        calibrator: Optional[Callable[[float], float]] = None
        if variant.calibrate:
            raw: List[float] = []
            y: List[int] = []
            for vc in train_videos:
                raw.extend(float(p) for p in
                           model.predict_proba(_feature_matrix(vc, variant.feature_names or
[])))
                y.extend(label_candidates(vc.intervals, vc.gts or [], iou))
            if variant.calibrate == "platt":
                calibrator = fit_platt(raw, y)
            elif variant.calibrate == "isotonic":
                calibrator = fit_isotonic(raw, y)
            else:
                raise ExperimentError(f"unknown calibrate={variant.calibrate!r} (use
'platt'/'isotonic')")
        threshold: Optional[float] = None
        if variant.tune_threshold:
            best_t, best_f1 = variant.threshold, -1.0
            for k in range(1, 20):                      # grid 0.05..0.95 on the train
fold's rally F1
                t = round(0.05 * k, 2)
                f1s = [score(predict(variant, vc, model=model, threshold=t,
calibrator=calibrator),
                             vc.gts or [], iou).f1 for vc in train_videos]
                mean_f1 = sum(f1s) / len(f1s) if f1s else 0.0
                if mean_f1 > best_f1:
                    best_f1, best_t = mean_f1, t
            threshold = best_t
        return calibrator, threshold
```

```
276
277
278     # ---------------------------------------------------------------------- variant scoring
279
280
281     def _train(variant: Variant, videos: Sequence[VideoCandidates], iou: float) ->
LogisticRegression:
282         """Fit the variant's classifier on the pooled candidates of ``videos`` (labels via
the
283         evaluator's own IoU matcher). The honest-LOVO training step from `distill_local`."""
284         X: List[List[float]] = []
285         y: List[int] = []
286         for vc in videos:
287             if vc.gts is None:
288                 raise ExperimentError(f"cannot train: {vc.name} has no golden labels")
289             X.extend(_feature_matrix(vc, variant.feature_names or []))
290             y.extend(label_candidates(vc.intervals, vc.gts, iou))
291         if not X:
292             raise ExperimentError("cannot train a learned variant on zero candidates")
293         return LogisticRegression().fit(X, y, variant.feature_names)
294
295
296     def evaluate_variant(videos: Sequence[VideoCandidates], variant: Variant,
297                          iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
298         """Score a variant on a labeled corpus. Returns per-video Scores + predictions + a
299         mean aggregate.
300
301         Prediction policy:
302           - **static variant** (no learned filter): predict each video directly ? no
training,
303             so no leakage; per-video scoring is already honest.
304           - **learned, pinned model** (``classifier_model`` set): score every video with the
305             SHIPPED model. ? optimistic if those videos were in its training set ? use this
to
306             report "how the shipped artifact does here", not for the promotion decision.
307           - **learned recipe** (``feature_names``, no pinned model) + ``honest=True``: LOVO
?
308             train on the OTHER videos, predict the held-out one. The number to trust.
309           - learned recipe + ``honest=False``: train on all, predict each (overfit; sanity
only).
310         """
311         for vc in videos:
312             if vc.gts is None:
313                 raise ExperimentError(f"{vc.name} has no golden labels; use
compare_unlabeled")
314
315         per_video: List[Dict[str, Any]] = []
316         predictions: Dict[str, List[Interval]] = {}
317
318         recipe_lovo = bool(variant.feature_names) and variant.classifier_model is None
319         pinned_model = (LogisticRegression.load(variant.classifier_model)
320                         if variant.classifier_model is not None else None)
321         all_model = None
322         if recipe_lovo and not honest:
323             all_model = _train(variant, videos, iou)
324
325         for i, vc in enumerate(videos):
326             cal: Optional[Callable[[float], float]] = None
327             thr: Optional[float] = None
328             if recipe_lovo and honest:
329                 others = [v for j, v in enumerate(videos) if j != i]
330                 if not others:
331                     raise ExperimentError("LOVO needs >=2 videos; pass honest=False or a
pinned model")
332                 model: Optional[LogisticRegression] = _train(variant, others, iou)
```

```
                if variant.tune_threshold or variant.calibrate:    # operating point from
the train fold
                    assert model is not None                       # _train never returns
None
                    cal, thr = _calibrate_and_tune(variant, model, others, iou)
            elif recipe_lovo:                          # honest=False ? one model over all
videos
                model = all_model
            else:                                      # static variant or pinned model
                model = pinned_model
            preds = predict(variant, vc, model=model, threshold=thr, calibrator=cal)
            assert vc.gts is not None                  # guarded at function top
            sc = score(preds, vc.gts, iou)
            per_video.append({"video": vc.name, "num_pred": len(preds), **{m: getattr(sc, m)
for m in _METRICS}})
            predictions[vc.name] = preds

        aggregate = {m: (sum(p[m] for p in per_video) / len(per_video) if per_video else
0.0)
                     for m in _METRICS}
        return {
            "variant": variant.name,
            "spec_hash": variant.spec_hash(),
            "is_learned": variant.is_learned(),
            "honest_lovo": recipe_lovo and honest,
            "per_video": per_video,
            "aggregate": aggregate,
            "predictions": predictions,
        }


    def _ci_block(eval_v: Dict[str, Any]) -> Dict[str, Any]:
        """Per-metric mean + bootstrap CI over the per-video scores (C1 ? never a bare
mean)."""
        return {m: _stats.mean_with_ci([p[m] for p in eval_v["per_video"]]) for m in
_METRICS}


    def _segmentation_block(videos: Sequence[VideoCandidates], eval_v: Dict[str, Any]) ->
Dict[str, Any]:
        """Mean F1@k + segment-count ratio across videos (C4 ? the over-segmentation tIoU
hides)."""
        gts_by_name = {v.name: (v.gts or []) for v in videos}
        overlaps = _segm.DEFAULT_OVERLAPS
        f1k: Dict[float, List[float]] = {ov: [] for ov in overlaps}
        ratios: List[float] = []
        for name, preds in eval_v.get("predictions", {}).items():
            gts = gts_by_name.get(name, [])
            got = _segm.f1_at_overlaps(preds, gts, overlaps)
            for ov in overlaps:
                f1k[ov].append(got[ov])
            ratios.append(_segm.segment_count_ratio(preds, gts))

        def _mean(xs: List[float]) -> float:
            return sum(xs) / len(xs) if xs else 0.0
        out: Dict[str, Any] = {f"f1@{int(ov * 100)}": _mean(vals) for ov, vals in
f1k.items()}
        out["segment_count_ratio"] = _mean(ratios)
        return out


    def honest_summary(videos: Sequence[VideoCandidates], eval_a: Dict[str, Any],
                       eval_b: Dict[str, Any]) -> Dict[str, Any]:
        """C1 + C4 honest reporting layered over a compare() pair (additive,
EXPERIMENT_HARNESS §6).
```

```
387
388         ``per_video`` lists are video-aligned (``evaluate_variant`` iterates ``videos`` in
order),
389         so ``paired_delta_f1`` pairs B?A by video ? the promotion-grade view: a lift is real
when
390         the ?F1 CI clears 0 *and* the sign test agrees, not when a bare mean ticked up.
391         """
392         a_f1 = [p["f1"] for p in eval_a["per_video"]]
393         b_f1 = [p["f1"] for p in eval_b["per_video"]]
394         return {
395             "variant_a_ci": _ci_block(eval_a),
396             "variant_b_ci": _ci_block(eval_b),
397             "paired_delta_f1": _stats.paired_delta_ci(a_f1, b_f1),
398             "segmentation": {"variant_a": _segmentation_block(videos, eval_a),
399                              "variant_b": _segmentation_block(videos, eval_b)},
400         }
401
402
403     def compare(videos: Sequence[VideoCandidates], variant_a: Variant, variant_b: Variant,
404                 iou: float = 0.5, honest: bool = True, honest_stats: bool = True) ->
Dict[str, Any]:
405         """Offline labeled A/B (`EXPERIMENT_HARNESS.md` §5.1). Scores both variants on the
same
406         golden corpus and reports the **B ? A** deltas, per video and in aggregate.
407
408         The deltas use the existing `metrics.score` (per video) +
`calibration.pct_improvement`;
409         learned variants are scored LOVO-honestly. The result is a self-contained record fit
for
410         `record_experiment` ? variant specs + hashes + the label-set fingerprint travel with
it.
411
412         ``honest_stats`` (default True) **adds** a ``"honest"`` block ? per-metric bootstrap
CIs, a
413         paired ?F1 CI + exact sign test (C1), and F1@k + segment-count ratio (C4) ?
*alongside* the
414         existing bare-mean fields (additive: nothing existing changes). Set False for the
legacy
415         shape. This is the measurement-honesty wiring (`ANALYZERS_AND_RECONCILERS.md`
§13/C1+C4): a
416         bare LOVO mean at n?6 is not trustworthy on its own.
417         """
418         if len(videos) < 1:
419             raise ExperimentError("compare needs at least one labeled video")
420         eval_a = evaluate_variant(videos, variant_a, iou, honest)
421         eval_b = evaluate_variant(videos, variant_b, iou, honest)
422
423         delta = {m: eval_b["aggregate"][m] - eval_a["aggregate"][m] for m in _METRICS}
424         delta_pct = {m: pct_improvement(eval_a["aggregate"][m], eval_b["aggregate"][m]) for
m in _METRICS}
425
426         by_video_a = {p["video"]: p for p in eval_a["per_video"]}
427         per_video_delta = [
428             {"video": p["video"], **{m: p[m] - by_video_a[p["video"]][m] for m in _METRICS}}
429             for p in eval_b["per_video"]
430         ]
431
432         # One fingerprint over the whole label set ? detects "the deltas were measured
against
433         # different labels" the same way regression.gt_hash guards the no-regression
baseline.
434         all_gts: List[Interval] = [iv for vc in videos for iv in (vc.gts or [])]
435
436         result: Dict[str, Any] = {
437             "iou": iou,
```

```
            "label_set_hash": gt_hash(all_gts),
            "num_videos": len(videos),
            "variant_a": eval_a,
            "variant_b": eval_b,
            "delta": delta,
            "delta_pct": delta_pct,
            "per_video_delta": per_video_delta,
        }
        if honest_stats:
            result["honest"] = honest_summary(videos, eval_a, eval_b)
        return result


    # ---------------------------------------------------------------- SxS on unlabeled video


    def compare_unlabeled(video: VideoCandidates, variant_a: Variant, variant_b: Variant,
                          agree_iou: float = 0.5) -> Dict[str, Any]:
        """Side-by-side on NEW footage with no ground truth (`EXPERIMENT_HARNESS.md` §5.2).

        With no labels there is no lift to measure ? instead this surfaces where the two
        variants **agree vs diverge**, which is exactly what a human reviews (and what two
        highlight reels would show side by side):

          - ``agreed``  ? windows both variants found (matched at ``agree_iou``);
          - ``a_only``  ? A fired, B suppressed (B's added precision, if A was over-firing);
          - ``b_only``  ? B fired, A did not (B's new detections ? real recall or new FPs:
            the human / a later golden label adjudicates).

        Both variants must be predictable without labels: a learned variant therefore needs
a
        pinned ``classifier_model`` (you cannot train on an unlabeled video). Reuses
        `metrics.match_intervals` ? no new matching logic.
        """
        preds_a = predict(variant_a, video)
        preds_b = predict(variant_b, video)
        mr = match_intervals(preds_a, preds_b, agree_iou)

        agreed = [{"a": preds_a[m.pred_index], "b": preds_b[m.gt_index], "iou": m.iou}
                  for m in mr.matches]
        agree_ious = [m.iou for m in mr.matches]
        a_only = [preds_a[i] for i in mr.unmatched_pred_indices]
        b_only = [preds_b[i] for i in mr.unmatched_gt_indices]

        def _dur(ivs: List[Interval]) -> float:
            return sum(e - s for s, e in ivs)

        return {
            "video": video.name,
            "agree_iou": agree_iou,
            "variant_a": variant_a.name,
            "variant_b": variant_b.name,
            "num_a": len(preds_a),
            "num_b": len(preds_b),
            "num_agreed": len(agreed),
            "mean_agree_iou": (sum(agree_ious) / len(agree_ious)) if agree_ious else 0.0,
            "duration_a": _dur(preds_a),
            "duration_b": _dur(preds_b),
            "agreed": agreed,
            "a_only": a_only,
            "b_only": b_only,
        }


    # ---------------------------------------------------------------------- leaderboard / DB
```

```


    def record_experiment(result: Dict[str, Any], path: str = DEFAULT_LEADERBOARD_PATH,
                          timestamp: Optional[str] = None) -> Dict[str, Any]:
        """Append a compact, reproducible record of a `compare()` result to the JSON
        leaderboard (`EXPERIMENT_HARNESS.md` §6: experiments are records). Generalises
        `regression_baseline.json`; deliberately the size of a baseline file, not a service
        (§9). Returns the row written. ``timestamp`` is injectable for deterministic tests.
        """
        row: Dict[str, Any] = {
            "timestamp": timestamp or datetime.now(timezone.utc).isoformat(),
            "iou": result.get("iou"),
            "label_set_hash": result.get("label_set_hash"),
            "num_videos": result.get("num_videos"),
            "variant_a": {"name": result["variant_a"]["variant"],
                          "spec_hash": result["variant_a"]["spec_hash"],
                          "aggregate": result["variant_a"]["aggregate"]},
            "variant_b": {"name": result["variant_b"]["variant"],
                          "spec_hash": result["variant_b"]["spec_hash"],
                          "aggregate": result["variant_b"]["aggregate"]},
            "delta": result.get("delta"),
        }
        # Record the honest ?F1 (CI + sign test) when present, so the leaderboard carries
the
        # promotion-grade evidence, not just the bare-mean delta (additive ? C1).
        honest = result.get("honest") or {}
        if honest.get("paired_delta_f1"):
            row["honest_delta_f1"] = honest["paired_delta_f1"]
        entries: List[Dict[str, Any]] = []
        if os.path.isfile(path):
            with open(path, encoding="utf-8") as f:
                entries = json.load(f)
        entries.append(row)
        os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
        with open(path, "w", encoding="utf-8") as f:
            json.dump(entries, f, indent=2)
        logger.info("recorded experiment %s vs %s (?f1=%.3f) -> %s",
                    row["variant_a"]["name"], row["variant_b"]["name"],
                    (result.get("delta") or {}).get("f1", 0.0), path)
        return row


    def load_video_candidates(db: Any, video_id: str, *, detector: Optional[str] = None,
                              gts: Optional[List[Interval]] = None,
                              feature_names: Sequence[str] = ()) -> VideoCandidates:
        """Bridge persisted detection ? `VideoCandidates` via
`Database.query_candidate_segments`.

        ``feature_names`` selects which persisted features to lift out of each row's
``stats``
        JSON (``"duration"`` is derived from the window bounds). This is the only DB-
touching
        function; the comparison itself is pure and offline. Exercised owner-side on a real
DB,
        but works against any `Database` (including an in-memory one in tests).
        """
        rows = db.query_candidate_segments(video_id, detector=detector)
        intervals: List[Interval] = [(float(r["start_time"]), float(r["end_time"])) for r in
rows]
        features: Dict[str, List[float]] = {}
        for name in feature_names:
            if name == "duration":
                features[name] = [float(r["end_time"]) - float(r["start_time"]) for r in
rows]
            else:
```

```
560                 features[name] = [float((r.get("stats") or {}).get(name, 0.0)) for r in
rows]
561         return VideoCandidates(name=video_id, intervals=intervals, features=features,
gts=gts)
562
```

6. assistant (2026-06-14T08:24:17.414Z)

Now let me read the fusion substrate files.

7. user (2026-06-14T08:24:18.099Z)

```
1       """Offline golden-set person-feature extractor ? the fusion-P3 measurement's GPU step.
2
3       Mirrors `distill_local`'s CSV-driven flow but ADDS the ~5 person-cue features per
shuttle
4       candidate window, by running `PersonDetector` over the ORIGINAL video. It reuses the
5       EXISTING golden trajectory CSVs (so **no WASB re-run**), needs **no Gemini and no DB**,
and
6       writes one small JSON per video. The LOVO comparison (`fusion_compare.py`) then runs
100% on
7       CPU offline. So the GPU touches exactly one thing ? person detection ? and the lift is
8       measured by the shipped experiment harness; person features enter only as feature
columns the
9       learned scorer weights (no special-casing, default-OFF until the golden ?F1 justifies
them).
10
11      Run (on the GPU box):
12          python -m backend.eval.fusion_golden --manifest output/human_lovo_manifest.json
--out-dir output
13
14      Each output `output/<name>_golden_features.json` is the replayable feature substrate ?
re-run
15      `fusion_compare` with different variants/thresholds forever without re-detecting on the
GPU.
16      """
17      from __future__ import annotations
18
19      import argparse
20      import json
21      import os
22      import time
23      from typing import Any, Dict, List, Optional
24
25      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
26      from backend.pipeline.detectors import rally_gate
27      from backend.pipeline.detectors import fusion_features as ff
28      from backend.eval import distill_local
29      from backend.eval.calibrate_wasb import load_golden_gts
30      from backend.eval.training import label_candidates
31
32      # The 5 fps/resolution-invariant shuttle features distill_local already computes.
33      SHUTTLE_FEATURE_NAMES = list(distill_local.FEATURE_NAMES)
34      # The 5 person-cue features (fusion_features.PERSON_FEATURE_NAMES).
35      PERSON_FEATURE_NAMES = list(ff.PERSON_FEATURE_NAMES)
36
37
38      def _derive_video_path(spec: Dict[str, Any]) -> str:
39          """Original MP4 for a manifest entry ? explicit `video_path`, else derived from the
40          golden labels path (``X.rallies.csv`` -> ``X.MP4``, the golden-set naming
convention)."""
41          if spec.get("video_path"):
42              return str(spec["video_path"])
43          labels = str(spec["labels"])
44          if labels.endswith(".rallies.csv"):
```

```
            return labels[: -len(".rallies.csv")] + ".MP4"
        raise ValueError(f"{spec.get('name')}: no video_path and labels {labels!r} isn't
*.rallies.csv")


    def extract_video(spec: Dict[str, Any], detector: Optional[Any] = None,
                      frame_skip: int = 3, iou: float = 0.5,
                      log_every_sec: float = 15.0) -> Dict[str, Any]:
        """One golden video -> a feature record (shuttle + person features per candidate
window).

        Returns ``{name, fps, frame_width,
candidates:[{start,end,shuttle{},person{},label}], gts}``.
        `detector` (anything with ``detections_over_windows``) is injectable for GPU-free
tests;
        when None a real `PersonDetector` is built (loads torchvision on first use).
        """
        fps, fw = float(spec["fps"]), float(spec["frame_width"])
        traj = str(spec["trajectory"])
        points = TrackNetRunner.parse_trajectory_csv(traj)
        intervals, x_shuttle = distill_local.build_candidates(traj, fps, fw)
        gts = load_golden_gts(str(spec["labels"]))
        labels = label_candidates(intervals, gts, iou)
        # Net axis for the person two-sided split ? reuse rally_gate's camera-agnostic
estimate
        # (parity with the shuttle net-crossing gate) rather than hard-coding an
orientation.
        axis, net_px, _span = rally_gate.estimate_play_axis(points)
        video_path = _derive_video_path(spec)

        if detector is None:
            from backend.pipeline.detectors.person_detector import PersonDetector
            detector = PersonDetector({"indexing": {"use_gpu": True}})

        # Single-pass person detection over ALL candidate windows (open the video ONCE, read
        # forward ? no per-window seek). Then compute features per window (pure, no GPU).
        n = len(intervals)
        win_frames = [(round(s * fps), round(e * fps)) for (s, e) in intervals]
        print(f"[fusion_golden]   {spec['name']}: {n} windows ? person-detecting in one
pass...", flush=True)
        t0 = time.monotonic()
        state = {"last": t0}

        def _progress(done: int, total: int) -> None:
            now = time.monotonic()
            if now - state["last"] >= log_every_sec or done >= total:
                el = now - t0
                rate = done / el if el > 0 else 0.0
                eta = (total - done) / rate if rate > 0 else 0.0
                pct = 100 * done // total if total else 0
                print(f"[fusion_golden]   {spec['name']}: detect frame {done}/{total}
({pct}%) "
                      f"| {el:.0f}s elapsed | ETA {eta:.0f}s", flush=True)
                state["last"] = now

        per_window = detector.detections_over_windows(video_path, win_frames, frame_skip,
                                                      progress_cb=_progress)

        candidates: List[Dict[str, Any]] = []
        for i, ((s, e), xs) in enumerate(zip(intervals, x_shuttle)):
            det = per_window[i]
            det_fw = det.get("frame_width") or fw
            det_fh = det.get("frame_height") or 0.0
            # Normalise the (pixel) net coord to 0-1 on the play axis, matching the foot-
point
```

```
            # normalisation fusion_features uses (x/width, y/height).
            if axis == "x":
                net_norm = net_px / det_fw if det_fw > 0 else 0.5
            else:
                net_norm = net_px / det_fh if det_fh > 0 else 0.5
            sf, ef = win_frames[i]
            shuttle_in = [p for p in points if sf <= p.frame <= ef]
            person = ff.compute_person_features(det["frames"], shuttle_in, det_fw, det_fh,
                                                court_polygon=None, net=(axis, net_norm))
            candidates.append({
                "start": float(s), "end": float(e),
                "shuttle": {k: float(v) for k, v in zip(SHUTTLE_FEATURE_NAMES, xs)},
                "person": {k: float(v) for k, v in person.items()},
                "label": int(labels[i]),
            })
        return {
            "name": spec["name"], "fps": fps, "frame_width": fw,
            "candidates": candidates,
            "gts": [[float(a), float(b)] for a, b in gts],
        }


    def run(manifest_path: str, out_dir: str, frame_skip: int = 3, iou: float = 0.5,
            fresh: bool = False, smallest_first: bool = False) -> List[str]:
        with open(manifest_path, encoding="utf-8") as f:
            specs = json.load(f)
        if smallest_first:
            # Process small videos first for quick feedback (the slow full match lands
last);
            # the LOVO result is order-independent. Proxy size by the trajectory CSV (one
row/frame).
            specs = sorted(specs, key=lambda s: (os.path.getsize(s["trajectory"])
                                                 if os.path.isfile(s.get("trajectory", ""))
else 0))
        os.makedirs(out_dir, exist_ok=True)
        written: List[str] = []
        for vi, spec in enumerate(specs, 1):
            out = os.path.join(out_dir, f"{spec['name']}_golden_features.json")
            # Resumable: skip a video whose JSON already exists (a re-run picks up where a
prior
            # run left off instead of re-detecting). --fresh forces a redo.
            if not fresh and os.path.isfile(out):
                print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: already
extracted "
                      f"-> {out} (use --fresh to redo)", flush=True)
                written.append(out)
                continue
            print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: extracting...",
flush=True)
            t0 = time.monotonic()
            data = extract_video(spec, frame_skip=frame_skip, iou=iou)
            n_pos = sum(c["label"] for c in data["candidates"])
            with open(out, "w", encoding="utf-8") as f:
                json.dump(data, f, indent=2)
            print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: DONE in
{time.monotonic() - t0:.0f}s "
                  f"| {len(data['candidates'])} windows ({n_pos} positive) |
{len(data['gts'])} golden rallies "
                  f"-> {out}", flush=True)
            written.append(out)
        return written


    def main() -> None:
        ap = argparse.ArgumentParser(description="Extract golden-set shuttle+person features
```

```
(fusion P3 GPU step).")
158         ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
159         ap.add_argument("--out-dir", default="output")
160         ap.add_argument("--frame-skip", type=int, default=3)
161         ap.add_argument("--iou", type=float, default=0.5)
162         ap.add_argument("--fresh", action="store_true", help="re-extract even if a JSON
already exists")
163         ap.add_argument("--smallest-first", action="store_true",
164                         help="process smaller videos first for quick feedback (LOVO is
order-independent)")
165         args = ap.parse_args()
166         run(args.manifest, args.out_dir, args.frame_skip, args.iou, fresh=args.fresh,
167             smallest_first=args.smallest_first)
168
169
170     if __name__ == "__main__":
171         main()
172
```

8. user (2026-06-14T08:24:18.242Z)

```
1       """Pure per-window person-cue features for multi-signal rally fusion
(MULTI_SIGNAL_FUSION_PLAN §3.2/§4).
2
3       NO I/O, no torch, no cv2 ? mirrors `rally_gate.py`'s discipline so the whole feature
4       math is unit-testable in a GPU-free dev container. The detection (person boxes,
5       shuttle trajectory) is produced elsewhere (`PersonDetector`, the WASB runner); this
6       module only turns those signals into a handful of **scale/translation-invariant**
7       features (counts / ratios / zone-membership ? never raw pixel coordinates, which would
8       memorise this court/camera and fail to generalise off a small labelled set).
9
10      Conventions:
11      - A person box is ``(x1, y1, x2, y2)`` in **pixels**; ``(track_id, box)`` when motion
needs it.
12      - ``per_frame`` is ``{frame_idx: [(track_id, box), ...]}`` keyed by the ORIGINAL video
frame
13        index (so it aligns directly with the shuttle trajectory's ``.frame`` ? no clip re-
timing).
14      - A shuttle point has ``.frame``, ``.visible``, ``.x``, ``.y`` (pixels) ? the WASB
trajectory shape.
15      - ``court_polygon`` is normalised 0-1 ``[(x, y), ...]`` or ``None`` (None ? no mask,
every
16        detection counts ? the player-count-only mode).
17      - ``net`` is ``(axis 'x'|'y', position 0-1 normalised)`` or ``None`` (None ? two-sided =
0.0).
18
19      Every function degrades gracefully (returns 0.0 / empty) on missing inputs; outputs are
20      floats in [0, 1] except ``mean_player_motion`` (a non-negative normalised speed).
21      """
22      from __future__ import annotations
23
24      from typing import Dict, List, Optional, Sequence, Tuple
25
26      BBox = Tuple[float, float, float, float]
27      Net = Tuple[str, float]
28
29
30      def point_in_polygon(point: Tuple[float, float], polygon: Sequence[Tuple[float, float]])
-> bool:
31          """Ray-casting point-in-polygon. `point` and `polygon` share a coordinate space
32          (we use normalised 0-1). Empty/degenerate polygon (<3 vertices) ? False."""
33          if polygon is None or len(polygon) < 3:
34              return False
35          x, y = point
36          inside = False
```

```
        n = len(polygon)
        j = n - 1
        for i in range(n):
            xi, yi = polygon[i]
            xj, yj = polygon[j]
            # Does the horizontal ray at y cross edge (i, j)?
            if (yi > y) != (yj > y):
                x_cross = (xj - xi) * (y - yi) / (yj - yi) + xi if yj != yi else xi
                if x < x_cross:
                    inside = not inside
            j = i
        return inside


    def _foot_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
        """Normalised foot point (bottom-centre) of a person box ? where the player stands,
        the right anchor for court membership. Returns (0,0) if frame dims are unusable."""
        if frame_width <= 0 or frame_height <= 0:
            return (0.0, 0.0)
        x1, _y1, x2, y2 = box
        return ((x1 + x2) / 2.0 / frame_width, y2 / frame_height)


    def _center_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
        if frame_width <= 0 or frame_height <= 0:
            return (0.0, 0.0)
        x1, y1, x2, y2 = box
        return ((x1 + x2) / 2.0 / frame_width, (y1 + y2) / 2.0 / frame_height)


    def person_in_court(box: BBox, frame_width: float, frame_height: float,
                        court_polygon: Optional[Sequence[Tuple[float, float]]]) -> bool:
        """Is this person on the court? Foot-point-in-polygon when a polygon is supplied;
        True (count everyone) when it is None. Unusable frame dims (w/h <= 0) make the foot
        point meaningless, so a masked check returns False rather than testing a bogus
(0,0)."""
        if court_polygon is None:
            return True
        if frame_width <= 0 or frame_height <= 0:
            return False
        return point_in_polygon(_foot_norm(box, frame_width, frame_height), court_polygon)


    def in_court_counts(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
                        frame_height: float,
                        court_polygon: Optional[Sequence[Tuple[float, float]]]) ->
List[int]:
        """Per-frame count of in-court persons (one entry per sampled frame)."""
        return [
            sum(1 for _tid, box in dets if person_in_court(box, frame_width, frame_height,
court_polygon))
            for _frame_idx, dets in sorted(per_frame.items())
        ]


    def _median(vals: Sequence[float]) -> float:
        if not vals:
            return 0.0
        s = sorted(vals)
        n = len(s)
        return float(s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2]))


```

```
    def players_in_court(counts: Sequence[int]) -> float:
        """Median per-frame in-court count (?2 singles / 4 doubles; 0-1 = dead time)."""
        return _median(list(counts))


    def in_court_ratio(counts: Sequence[int], min_players: int = 2) -> float:
        """Fraction of sampled frames with >= `min_players` in court (track stability vs
flicker)."""
        if not counts:
            return 0.0
        return sum(1 for c in counts if c >= min_players) / len(counts)


    def mean_player_motion(per_frame: Dict[int, List[Tuple[int, BBox]]],
                           frame_width: float, frame_height: float) -> float:
        """Mean per-track centre displacement between consecutive sampled frames, with x
        normalised by frame width and y by frame height (per-axis; uniform-scale-invariant,
        not aspect-ratio-isotropic). 0.0 if <2 frames or no shared tracks."""
        if frame_width <= 0 or len(per_frame) < 2:
            return 0.0
        frames = sorted(per_frame.keys())
        steps: List[float] = []
        for a, b in zip(frames, frames[1:]):
            prev = {tid: box for tid, box in per_frame[a]}
            for tid, box in per_frame[b]:
                if tid in prev:
                    cx0, cy0 = _center_norm(prev[tid], frame_width, frame_height)
                    cx1, cy1 = _center_norm(box, frame_width, frame_height)
                    steps.append(((cx1 - cx0) ** 2 + (cy1 - cy0) ** 2) ** 0.5)
        if not steps:
            return 0.0
        return sum(steps) / len(steps)


    def two_sided_motion(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
                         frame_height: float, net: Optional[Net],
                         court_polygon: Optional[Sequence[Tuple[float, float]]]) -> float:
        """Fraction of sampled frames with >=1 in-court person on BOTH net halves.

        A real rally is two-sided; a single player feeding/holding the shuttle is one-sided.
        The net splits the play axis at `net=(axis, position)` in normalised coords. None ?
0.0.
        """
        if net is None or not per_frame:
            return 0.0
        axis, pos = net
        both = 0
        for _frame_idx, dets in per_frame.items():
            near, far = False, False
            for _tid, box in dets:
                if not person_in_court(box, frame_width, frame_height, court_polygon):
                    continue
                fx, fy = _foot_norm(box, frame_width, frame_height)
                coord = fx if axis == "x" else fy
                if coord < pos:
                    near = True
                else:
                    far = True
            if near and far:
                both += 1
        return both / len(per_frame)


    def _shuttle_in_box(sx: float, sy: float, box: BBox, frame_width: float, frame_height:
float,
```

```
                        pad_frac: float) -> bool:
        """Is the (pixel) shuttle point inside a person box, expanded by `pad_frac` of frame
        width/height (the 'adjacent / in-hand' tolerance)?"""
        x1, y1, x2, y2 = box
        px = pad_frac * frame_width
        py = pad_frac * frame_height
        return (x1 - px) <= sx <= (x2 + px) and (y1 - py) <= sy <= (y2 + py)


    def shuttle_player_colocation(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
                                  frame_width: float, frame_height: float,
                                  pad_frac: float = 0.03) -> float:
        """Fraction of (visible shuttle frames that were also person-sampled) in which the
        shuttle sits inside/adjacent to a person box. **held/in-hand ? high; in-flight ?
~0**
        ? the direct held-shuttle discriminator. 0.0 if no aligned visible shuttle frames.
        """
        if not per_frame or frame_width <= 0 or frame_height <= 0:
            return 0.0
        aligned = 0
        colocated = 0
        for p in shuttle_points or []:
            if not getattr(p, "visible", False):
                continue
            dets = per_frame.get(int(p.frame))
            if dets is None:          # shuttle frame wasn't person-sampled ? can't judge
                continue
            aligned += 1
            if any(_shuttle_in_box(p.x, p.y, box, frame_width, frame_height, pad_frac)
                   for _tid, box in dets):
                colocated += 1
        return colocated / aligned if aligned else 0.0


    # The ~5 person features persisted into candidate `stats` (order = the learned-fusion
    # feature columns appended after the shuttle features in distill_local.FEATURE_NAMES).
    PERSON_FEATURE_NAMES = [
        "players_in_court", "two_sided_motion", "mean_player_motion",
        "in_court_ratio", "shuttle_player_colocation",
    ]


    def compute_person_features(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
                                frame_width: float, frame_height: float,
                                court_polygon: Optional[Sequence[Tuple[float, float]]] =
None,
                                net: Optional[Net] = None,
                                min_players: int = 2) -> Dict[str, float]:
        """Aggregate the per-window person features from per-frame detections + the in-
window
        shuttle trajectory. Pure; returns exactly `PERSON_FEATURE_NAMES`, all floats."""
        counts = in_court_counts(per_frame, frame_width, frame_height, court_polygon)
        return {
            "players_in_court": players_in_court(counts),
            "two_sided_motion": two_sided_motion(per_frame, frame_width, frame_height, net,
court_polygon),
            "mean_player_motion": mean_player_motion(per_frame, frame_width, frame_height),
            "in_court_ratio": in_court_ratio(counts, min_players),
            "shuttle_player_colocation": shuttle_player_colocation(
                per_frame, shuttle_points, frame_width, frame_height),
        }

```

9. assistant (2026-06-14T08:24:21.760Z)

Now let me look at the correction loop, the production pipeline integration, and how candidates are persisted, to understand the per-user label source and what's wired into production.

10. user (2026-06-14T08:24:22.627Z)

```
1       # Experiment harness ? A/B + shadow eval for rally-detection quality (no-regression by
design)
2
3       **Status:** **P1 IMPLEMENTED** (`backend/eval/experiment.py` +
`tests/test_experiment.py`, shipped
4       2026-06-13) ? the `Variant` layer, `compare()` (labeled A/B, LOVO-honest),
`compare_unlabeled()`
5       (SxS on NEW unlabeled footage), and the leaderboard. **P2?P4 remain owner-gated.**
Phases below are
6       sequenced but each is a separate PR.
7
8       > **Companion docs:** the cues this experiments *on* are in
9       > [MULTI_SIGNAL_FUSION_PLAN.md](MULTI_SIGNAL_FUSION_PLAN.md); the scoring mechanics are
10      > [EVALUATION.md](EVALUATION.md); the no-regression gate + golden labels +
`baseline.json` come from
11      > [GOLDEN_SET_VENDORING.md](GOLDEN_SET_VENDORING.md); the modular-cue architecture is
**ADR-007**
12      > ([archives/decisions/DECISIONS.md](archives/decisions/DECISIONS.md)); the distilled
classifier is
13      > **ADR-004**. The 2-layer mental model is
[HOW_RALLY_DETECTION_WORKS.md](HOW_RALLY_DETECTION_WORKS.md).
14
15      ---
16
17      ## 1. Context ? why this, why now
18
19      PR #112 (merged) added the multi-signal fusion design ? shuttle + person + audio cues,
all **default
20      OFF**. The owner's directive for *how* we evolve quality from here:
21
22      > *"Evolve the codebase so we can keep finding new ways to improve quality and
experiment with them ?
23      > or add them as a signal ? without suffering any regressions if the idea doesn't work
out. Rolling
24      > back the code would be too dangerous."*
25
26      So **experimentation and deployment must be decoupled.** The contract:
27
28      1. Merge experimental code freely ? it's **gated OFF**, so it cannot change production
behavior.
29      2. Test it **offline against golden labels** (most experiments never touch the served
path at all).
30      3. Promote a variant to default **only on measured lift**, through a CI eval gate.
31      4. **"Rollback" = flip a config flag, never `git revert`.** The proven path is a pinned
variant that is
32         always present.
33
34      **The pleasant surprise (Explore audit, 2026-06-13):** ~80% of this harness already
exists and is
35      simply not composed *as a system*. The missing piece is a thin **variant/experiment
layer + the
36      discipline**, not a platform. §4 is the exact code surface so the next session does not
re-discover it.
37
38      ---
39
40      ## 2. The thesis ? separate expensive DETECTION from cheap DECISION
41
```

```
42      The expensive, GPU/WSL-bound, risky part is **detection**: "what signals exist per
frame" (shuttle
43      WASB trajectory, person tracks/boxes, audio hits). The cheap, swappable, pure-Python
part is the
44      **decision**: "given those signals, where are the rallies" (windowing + gates + the
classifier).
45
46      > **Run detection once per video, persist all per-cue signals + per-candidate features,
then re-score
47      > any number of variants OFFLINE ? deterministically, with no GPU and zero production
risk.**
48
49      This is not aspirational ? `distill_local.py` and `calibrate_local.py` already do
exactly this on
50      stored WASB trajectories. The candidate-segments table (`stats` JSON) is the persistence
substrate.
51      ? Most experiments are pure re-scoring of stored data; they never enter the served
pipeline.
52
53      ---
54
55      ## 3. What already exists (the audit ? do NOT rebuild)
56
57      Exact public surface, with `file:line`. **This table is the anti-duplicate-work
artifact** ? start here.
58
59      ### 3.1 Scoring (variant-agnostic ? grades any `List[Interval]`)
60      `backend/eval/metrics.py`
61      - `interval_iou(a, b) -> float` (:15) ? temporal IoU of two `(start,end)` intervals.
62      - `match_intervals(preds, gts, iou_threshold=0.5) -> MatchResult` (:48) ? greedy 1-to-1
IoU matching.
63      - `score(preds, gts, iou_threshold=0.5, match_result=None) -> Score` (:104) ? P/R/F1,
mean IoU, boundary errors; `Score.as_dict()`.
64      - `pr_curve(preds, gts, thresholds=None) -> List[Score]` (:138).
65
66      ### 3.2 A/B + cross-validation primitives (the comparison engine already exists)
67      `backend/eval/calibration.py`
68      - `improvement_report(predict_fn, baseline_config, tuned_config, gts, iou_threshold=0.5)
-> dict` (:148) ? **before/after metrics + %?. This IS the A/B delta.**
69      - `leave_one_video_out(videos, configs, metric="f1", iou_threshold=0.5) -> dict` (:113)
? **LOVO: pick on other videos, score on held-out video (honest cross-video).**
70      - `cross_validate(predict_fn, configs, gts, k=5, metric="recall", iou_threshold=0.5) ->
dict` (:72) ? within-video k-fold overfit guard.
71      - `select_best(predict_fn, configs, gts, metric="f1", iou_threshold=0.5) -> (Config,
float)` (:61).
72      - `evaluate(preds, gts, iou_threshold=0.5) -> Score` (:41); `kfold_indices(n, k)` (:46).
73      - LOVO video dict shape: `{"name": str, "predict_fn": PredictFn, "gts": [Interval]}`.
74
75      ### 3.3 No-regression gate + baselines (the promotion gate already exists)
76      `backend/eval/regression.py`
77      - `regress(db, video_id, ground_truth, stage="final", iou_threshold=0.5, detector=None,
tolerance=DEFAULT_TOLERANCE, baseline_path=DEFAULT_BASELINE_PATH) -> RegressionResult` (:104) ?
score stored preds vs GT, flag if P **or** R drops > tolerance.
78      - `regress_with_provider(...)` (:160) ? same, fetching GT via the annotation provider.
79      - `save_baseline(video_id, stage, precision, recall, f1, ..., gt_fingerprint=None)`
(:80) · `load_baselines(path) -> dict` (:68).
80      - `RegressionResult` (:43): `regressed`, `reasons`, `gt_hash`, baseline P/R,
`tolerance`, `has_baseline`; `as_dict()`.
81      - Default baseline file: `eval_baselines/regression_baseline.json` (:33).
82
83      `run_regression.py` ? CI-gatable CLI, `main(argv)` (:52). **Exit codes: 0 ok / 1
regression / 2 no-DB / 3 no-baseline.** Flags: `--video-id --tier --annotations
--stage{candidates,final} --detector --iou --tolerance --baseline --update-baseline --allow-
missing-baseline --json`.
84
```

3.4 Pinnable model artifact
`backend/eval/classifier.py` ? `LogisticRegression` (pure numpy, L2): `fit(X,y,feature_names=,class_weight="balanced")` (:40), `predict_proba` (:75), `predict(threshold=0.5)` (:81), `to_dict`/`from_dict` (:86/:97), `save`/`load` (JSON) (:107/:111). **A trained model = one JSON file ? a pinnable, hashable variant artifact.**

3.5 Offline re-scoring substrate (persist once, re-score forever)
`backend/infrastructure/database.py`
- `add_candidate_segment(video_id, detector, start_time, end_time, chunk_index=None, confidence=None, stats=None, detection_params=None) -> Optional[int]` (:240) ? `stats`/`detection_params` stored as JSON; `candidate_segments` table cols incl. `stats`, `detection_params`, `human_label`, `confirmed_segment_id` (schema :97).
- `query_candidate_segments(video_id, detector=None, only_unlabeled=False) -> List[dict]` (:277) ? returns rows with deserialized `stats`.

3.6 Label assignment + feature glue
`backend/eval/training.py`
- `label_candidates(candidate_intervals, gt_intervals, iou_threshold=0.5) -> List[int]` (:50) ? y-labels by IoU match to golden (same matcher as the evaluator).
- `build_training_set(candidate_rows, gt_intervals, iou_threshold=0.5, feature_fn=extract_yolo_features) -> (X, y, intervals)` (:64) ? pluggable `feature_fn`.

3.7 Existing LOVO drivers to mirror (not duplicate)
- `backend/eval/distill_local.py` ? `run(specs, iou=0.5, threshold=0.5, model_out=None)` (:98), CLI `python -m backend.eval.distill_local --manifest videos.json --model-out ...`; `FEATURE_NAMES = [duration, peak_velocity, mean_velocity, active_ratio, net_crossings]` (:40); already does LOVO over Gemini silver labels.
- `backend/eval/calibrate_local.py` ? `run(specs, iou=0.5, metric="f1")` (:77); `LocalConfig = (velocity_thresh, merge_gap, min_window_duration, min_crossings)`; grid + LOVO.
- `backend/eval/calibrate_wasb.py` ? `run(...)`, `load_golden_gts()`; `backend/eval/gt_loader.py::load_gt_intervals(csv_path, *, strict=True)` is THE canonical golden reader.

3.8 Registries + config knobs (variant selection)
- `backend/pipeline/segmenters/base.py` ? `SegmenterRegistry` (:35), singleton `segmenter_registry` (:63); registered: `motion`, `gemini`, `yolo_hybrid`, `tracknet_hybrid`, `wasb_hybrid`.
- `backend/annotations/base.py` ? `AnnotationProviderRegistry`, singleton `annotation_registry`; providers `file`, `auto`.
- `backend/config/models.py::IndexingConfig` ? `default_segmenter` (:65, default `"yolo_hybrid"`), `detector_model` (:72), `detector_score_threshold` (:73), `yolo_velocity_threshold` (:74), `yolo_frame_skip` (:75), `min_segment_duration` (:68), `dead_time_merge_gap` (:69), `motion_threshold` (:67). **No `min_crossings` knob yet** (see fusion P2).

3.9 Confirmed ABSENT (no duplication risk)
Explore found **no** existing experiment / variant / shadow / A-B *machinery* ? only a stray "A/B test"
aspiration comment in `backend/pipeline/detectors/base.py` and an "experiment E1" note in
`AUDIO_RALLY_DETECTION_PLAN.md`. The `regress()` + baseline path is the canonical comparison mechanism.

---

4. The variant abstraction (the one thin thing to add)

A **`Variant` = a declarative recipe**, not a code branch:

```
Variant = (segmenter_name, IndexingConfig overrides, enabled cues + per-cue params, classifier model path/hash)
```

JSON-serializable; selected via the existing `segmenter_registry` +

`IndexingConfig.default_segmenter`.
    Control = a *pinned, frozen* variant (recipe + model hash) that is always registered and is the
    default. Adding a new cue = a new Variant differing by one flag ? the control recipe is untouched.

    ## 5. Three experiment modes

    ### 5.1 Offline A/B (primary, label-driven, zero production risk)
    `compare(videos, variant_A, variant_B)` ? a thin driver composing **existing** functions:
    `query_candidate_segments()` (load persisted features) ? re-score each variant ? `calibration.improvement_report()`
    + `calibration.leave_one_video_out()` + `metrics.score()`, graded vs golden (`gt_loader.load_gt_intervals`).
    Reports per-video **and** LOVO-honest aggregate IoU/P/R/F1 deltas. **This is the A/B test, and it needs
    almost no new scoring code** ? only the glue.

    Honest reporting (default ON, additive): `compare(..., honest_stats=True)` also attaches a `"honest"`
    block ? per-metric bootstrap **CIs**, a paired **?F1 CI + exact sign test** (C1), and **F1@k +
    segment-count ratio** (C4) ? *alongside* the bare-mean fields (existing output unchanged; `honest_stats=False`
    restores the legacy shape). A bare LOVO mean at n?6 is not promotable on its own; the lift is real when the
    ?F1 CI clears 0 **and** the sign test agrees. Wiring of the course corrections in
    ANALYZERS_AND_RECONCILERS.md §13/C1+C4 (helpers: `backend/eval/eval_stats.py`,
    `segmentation_metrics.py`); `record_experiment` carries the honest ?F1 into the leaderboard.

    ### 5.2 Shadow (for cues that touch detection, e.g. the person detector)
    The new cue runs and **persists its signal** into the candidate `stats`/`detection_params`, but the
    served decision stays control. Offline you then ask "what would variant B have scored?" via §5.1 ?
    B never affects output until it wins. This is how a detection-touching change (not just a re-scoring
    change) earns its way in without risk.

    ### 5.3 Promotion gate (no-regression, CI-enforced)
    `regression.regress()` + `baseline.json` + `run_regression.py` exit codes. A variant becomes the new
    default **only if** it doesn't regress the golden set beyond `tolerance` (exit 1 blocks). Control is
    pinned ? **rollback = flip `default_segmenter`/variant config, never a code revert.**

    ## 6. No-regression guarantees (the heart of the owner's ask)

    - **New cues/signals default OFF** ? per-cue enable flag in `IndexingConfig`; merged code can't change
      behavior until explicitly enabled.
    - **Control variant pinned** ? frozen recipe + classifier model hash, always registered & default.
    - **Promotion only via the eval gate** ? `run_regression.py` in CI (exit 1 blocks); no silent default change.
    - **Experiments are records** ? variant-spec hash ? per-video + LOVO scores + label-set hash + timestamp,
      persisted to a small JSON **experiment leaderboard** (generalizes `baseline.json`).

Answers "which
167   signals ever helped, on which video slices," and makes every result reproducible.
168   - **Decoupled events** ? "merge an experiment" and "change the default" are separate,
independently
169   revertible actions.
170
171   ---
172
173   ## 7. What to build later (gap list ? thin, additive, owner-gated)
174
175   1. ~~**`backend/eval/experiment.py`** ? a `Variant` dataclass + `compare(videos, a, b)`
composing the §3
176   functions (no new scoring math).~~ **DONE (2026-06-13).** Shipped `Variant` (+ pinned
`CONTROL`),
177   `compare` (labeled A/B with LOVO-honest learned-variant training, mirroring
`distill_local`),
178   `compare_unlabeled` (the **SxS-on-new-videos** mode ? agreed / A-only / B-only via
`match_intervals`,
179   no GT), `record_experiment` (? `eval_baselines/experiment_leaderboard.json`), and
`load_video_candidates`
180   (the `query_candidate_segments` bridge). JSON variant recipes first as recommended; a
`VariantRegistry`
181   only if the count grows. 19 tests incl. the no-regression invariant.
182   2. **Persist per-cue features** into candidate `stats` (person: `players_in_court`,
`two_sided_motion`,
183   `in_court_ratio`, `shuttle_player_colocation`; shuttle-state: `net_crossings`,
`direction_reversals`;
184   audio: hit times) so any future variant re-scores offline ? ties to
`MULTI_SIGNAL_FUSION_PLAN.md` §3.2/§4.
185   3. **Per-cue enable flags** in `IndexingConfig` (default `False`); `default_segmenter`
already exists.
186   Add the `min_crossings` knob (fusion P2) here too.
187   4. **CI promotion lane** using `run_regression.py` against the golden `baseline.json`;
write the
188   experiment-leaderboard JSON.
189
190   ---
191
192   ## 8. Phasing (owner-gated; ONE PR per slice, off `master`)
193
194   - **P0 ? this doc**, indexed in `docs/README.md`, cross-linked from
`MULTI_SIGNAL_FUSION_PLAN.md` and
195   `GOLDEN_SET_VENDORING.md`. *(this deliverable; docs-only, no code.)*
196   - **P1 ? `experiment.py` `compare()`** ? **DONE (2026-06-13):** thin wrapper over
`calibration` +
197   `metrics` + `query_candidate_segments`; pure-logic, fully tested in this dev container
(`compare`,
198   `compare_unlabeled`, `record_experiment`, the no-regression invariant).
`compare_unlabeled` adds the
199   SxS read on **new unlabeled footage** the owner asked for (run control + candidate on
a fresh video,
200   diff where they agree/diverge ? two reels via the existing `VideoCompiler` is the GPU-
side follow-on).
201   - **P2 ? per-cue enable flags + feature persistence** into candidate `stats` (default
OFF) ? couples
202   with fusion P1/P2.
203   - **P3 ? CI promotion gate**: `run_regression.py` lane on the golden `baseline.json`;
experiment-leaderboard JSON.
204
205   ---
206
207   ## 9. Scope guardrail
208
209   **Single-tenant / local-first** (per `PLATFORM_ARCHITECTURE.md`): a JSON experiment
registry + the

```
210     existing harness ? explicitly **NOT** MLflow / W&B / a service. Keep it the size of
`baseline.json`.
211
212     ---
213
214     ## 10. Verification
215
216     - **Offline, runnable in this dev container (no GPU/torch needed for the glue):** unit
test that
217       `compare()` is deterministic on synthetic candidate rows; a **no-regression invariant
test** asserting
218       "all new-cue flags OFF ? predictions byte-identical to control"; a CI-gate test
asserting
219       `run_regression.py` exit codes (0 ok / 1 regress / 3 no-baseline).
220     - **Machine-side (owner, GPU/WSL):** persist candidates once on a real clip, run
`compare(control,
221       fusion)` LOVO, confirm the reported lift matches a manual `metrics.score()` spot-
check.
222     - **Docs (P0):** links resolve; indexed in `docs/README.md`; cross-linked from
223       `MULTI_SIGNAL_FUSION_PLAN.md` and `GOLDEN_SET_VENDORING.md`.
224     </content>
225
```

11. user (2026-06-14T08:24:23.452Z)

```
Found 23 files
docs\QUALITY_ITERATIONS.md
docs\archives\quality-iterations\2026-06-first-ab-experiment\README.md
docs\EXPERIMENT_HARNESS.md
backend\eval\score_calibration.py
backend\eval\segmentation_metrics.py
tests\test_eval_stats.py
tests\test_score_calibration.py
tests\test_segmentation_metrics.py
backend\eval\eval_stats.py
docs\ANALYZERS_AND_RECONCILERS.md
backend\tools\export.py
docs\STORAGE_SHARING_MODEL.md
CLAUDE.md
docs\VIDEO_LOCALITY_MODEL.md
docs\PLATFORM_ARCHITECTURE.md
docs\COMMERCIALIZATION.md
docs\UI_ALPHA_EXECUTION_PLAN.md
docs\mockups\ui_alpha_mockup.html
docs\archives\decisions\DECISIONS.md
docs\UI_PROPOSAL.md
docs\archives\checkpoints\2026-06-strategy-foundation\README.md
docs\archives\COMMERCIAL_READINESS_REVIEW.md
docs\archives\MONETIZATION_AUDIT.md
```

12. assistant (2026-06-14T08:24:25.779Z)

Let me examine the correction loop (the per-user label source) and how candidates/human labels are persisted and consumed.

13. user (2026-06-14T08:24:27.414Z)

```
backend\infrastructure\database.py:109:                         confirmed_segment_id INTEGER,
backend\infrastructure\database.py:110:                         human_label TEXT,
backend\infrastructure\database.py:114:                         FOREIGN KEY
(confirmed_segment_id) REFERENCES play_segments(id) ON DELETE SET NULL
backend\infrastructure\database.py:279:                     "UPDATE candidate_segments SET
confirmed_segment_id = ? WHERE id = ?",
backend\infrastructure\database.py:299:            query += " AND human_label IS NULL"
```

```
backend\reporting\harvest.py:159:            st.add_metric("confirmed_segments",
len(play_segments))
backend\tools\export.py:6:machine-readable foundation for later human correction / golden
feeding
backend\eval\eval_stats.py:1:"""Honest small-N evaluation statistics (course-corrections C1 +
C3).
backend\eval\segmentation_metrics.py:1:"""Segmentation-aware metrics (course-correction C4) ?
make over-segmentation visible.
backend\pipeline\segmenters\yolo_hybrid.py:351:                    # may yield multiple AI
segments (one-to-many); confirmed_segment_id keeps
backend\eval\score_calibration.py:1:"""Per-cue score calibration (course-correction C2) ? make
``confidence`` mean something.
```

14. user (2026-06-14T08:24:31.892Z)

```
107-                        stats TEXT,
108-                        detection_params TEXT,
109-                        confirmed_segment_id INTEGER,
110:                        human_label TEXT,
111-                        notes TEXT,
112-                        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
113-                        FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
--
296-            query += " AND detector = ?"
297-            params.append(detector)
298-        if only_unlabeled:
299:            query += " AND human_label IS NULL"
300-
301-        query += " ORDER BY start_time"
302-
```

15. user (2026-06-14T08:24:33.510Z)

```
backend\tools\rally_slicer.py:53:def load_rally_labels(csv_path: str) -> Dict[str, Tuple[float,
float]]:
backend\tools\rally_slicer.py:76:def compute_clip_windows(labels: Dict[str, Tuple[float,
float]],
backend\tools\rally_slicer.py:118:def slice_rallies(video_path: str, labels_csv: str,
rally_numbers: List[str],
backend\eval\calibrate_wasb.py:66:def run(trajectory_csv: str, labels_csv: str, fps: float,
frame_width: float,
backend\tools\export.py:156:def segments_to_edl(segments: Iterable[Dict[str, Any]], title: str =
"Highlights") -> str:
backend\reporting\harvest.py:162:def _highlights(rec: "obsreport.RunRecorder", play_segments:
List[Dict[str, Any]],
backend\eval\audio\e1_probe.py:53:def run(video: str, labels_csv: str, trajectory: str, fps:
float, frame_width: float,
backend\annotations\automated_provider.py:67:    def __init__(self, labeler: Optional[Labeler] =
None, lineage: str = "unspecified"):
backend\eval\eval_partial_labels.py:68:def run(trajectory_csv: str, labels_csv: str, fps: float,
frame_width: float,
backend\eval\eval_partial_labels.py:142:def sweep(trajectory_csv: str, labels_csv: str, fps:
float, frame_width: float,
backend\utils\run_telemetry.py:128:    def __init__(self, path: str, label: str = "run",
max_bytes: int = 10_000,
backend\eval\export_wasb_segments.py:148:    def _gate(segs: List[Interval], label: str) ->
List[Interval]:
backend\eval\experiment.py:454:def compare_unlabeled(video: VideoCandidates, variant_a: Variant,
variant_b: Variant,
backend\main.py:448:def download_highlight(video_id: str, filename: str):
backend\main.py:483:def compile_highlights(req: CompileRequest):
backend\eval\training.py:50:def label_candidates(candidate_intervals: List[Interval],
gt_intervals: List[Interval],
backend\eval\score_calibration.py:71:def fit_platt(scores: Sequence[float], labels:
Sequence[int],
```

```
backend\eval\score_calibration.py:126:def fit_isotonic(scores: Sequence[float], labels:
Sequence[int]) -> IsotonicCalibrator:
backend\eval\score_calibration.py:137:def brier_score(probs: Sequence[float], labels:
Sequence[int]) -> float:
backend\eval\score_calibration.py:145:def expected_calibration_error(probs: Sequence[float],
labels: Sequence[int],
backend\eval\score_calibration.py:169:def calibration_report(probs: Sequence[float], labels:
Sequence[int],
backend\eval\rally_seq_proto.py:53:def labels_for(times: np.ndarray, gts: List[Interval]) ->
np.ndarray:
backend\pipeline\stitcher\compiler.py:163:    def compile_highlights(self, raw_video_path: str,
segments: List[Tuple[float, float]], output_filename: str) -> str:
```

16. assistant (2026-06-14T08:24:37.198Z)

Now let me understand how the correction loop / human label feeds back, and whether there's any per-user / per-tenant model store. Let me check the API endpoints and the play_segments confirmation flow, plus any multi-user/tenant concept.

17. user (2026-06-14T08:24:39.224Z)

```
backend\config\models.py:207:    # When set, /api/process video_path must resolve under this
directory (hosted/tenant
backend\utils\paths.py:42:    allowed_root (a future hosted/tenant deployment) enforces
containment.
```

18. user (2026-06-14T08:24:40.771Z)

```
No matches found
```

19. assistant (2026-06-14T08:24:44.354Z)

The grep for confirmation flow in main.py returned nothing. Let me find where human labels / corrections actually get written, and check the production segmenter to confirm whether the gate/fusion features are wired in.

20. user (2026-06-14T08:24:45.921Z)

```
Found 9 files
backend\main.py
backend\infrastructure\database.py
backend\pipeline\segmenters\trajectory_hybrid.py
backend\pipeline\segmenters\yolo_hybrid.py
backend\eval\harness.py
backend\pipeline\segmenters\motion.py
backend\pipeline\segmenters\gemini.py
backend\reporting\harvest.py
backend\reporting\spec.yaml
```

21. user (2026-06-14T08:24:46.632Z)

```
240                with self._connect() as conn:
241                    conn.cursor().execute(
242                        "INSERT INTO events (segment_id, timestamp, type, player, details)
VALUES (?, ?, ?, ?, ?)",
243                        (segment_id, timestamp, event_type, player, json.dumps(details or
{}))
244                    )
245                    conn.commit()
246                return True
247            except Exception as e:
248                logger.error(f"Failed to add event: {e}")
249                return False
250
```

```
        def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
                                  chunk_index: Optional[int] = None, confidence:
Optional[float] = None,
                                  stats: Optional[Dict[str, Any]] = None,
                                  detection_params: Optional[Dict[str, Any]] = None) ->
Optional[int]:
            """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
            detector-specific dicts stored as JSON. Returns the new candidate id, or
None."""
            try:
                with self._connect() as conn:
                    cursor = conn.cursor()
                    cursor.execute(
                        """INSERT INTO candidate_segments
                           (video_id, detector, chunk_index, start_time, end_time, duration,
                            confidence, stats, detection_params)
                           VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
                        (video_id, detector, chunk_index, start_time, end_time, end_time -
start_time,
                         confidence, json.dumps(stats or {}), json.dumps(detection_params or
{}))
                    )
                    conn.commit()
                    return cursor.lastrowid
            except Exception as e:
                logger.error(f"Failed to add candidate segment: {e}")
                return None

        def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
            """Record that a candidate detection was confirmed as a given play_segment."""
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id =
?",
                        (segment_id, candidate_id)
                    )
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to link candidate {candidate_id} to segment
{segment_id}: {e}")
                return False

        def query_candidate_segments(self, video_id: str, detector: Optional[str] = None,
                                     only_unlabeled: bool = False) -> List[Dict[str, Any]]:
            """Fetch candidate detections for the annotation / fine-tuning workflow.
            `stats` and `detection_params` are deserialized from JSON."""
            query = "SELECT * FROM candidate_segments WHERE video_id = ?"
            params: List[Any] = [video_id]

            if detector:
                query += " AND detector = ?"
                params.append(detector)
            if only_unlabeled:
                query += " AND human_label IS NULL"

            query += " ORDER BY start_time"

            candidates = []
            with self._connect() as conn:
                rows = conn.cursor().execute(query, params).fetchall()
                for row in rows:
                    d = dict(row)
```

```
308                     d["stats"] = json.loads(d["stats"] or "{}")
309                     d["detection_params"] = json.loads(d["detection_params"] or "{}")
```

22. assistant (2026-06-14T08:24:50.280Z)

Let me check the correction loop in main.py and how human labels get set, plus whether fusion features are persisted in production (the gap between eval substrate and served pipeline).

23. user (2026-06-14T08:24:52.286Z)

```
52:os.makedirs("backend/api/static", exist_ok=True)
53:app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")
57:    index_path = "backend/api/static/index.html"
61:    return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
backend/api/static/</h3>"
119:@app.get("/api/config")
126:    This is the former synchronous /api/process body, unchanged in behavior, now run on
257:@app.post("/api/process", status_code=202)
259:    """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
286:        content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
290:@app.get("/api/jobs/{job_id}")
314:# under security.upload_dir, then submitted to /api/process by the returned path.
351:@app.post("/api/upload/init")
373:            "next_part": f"/api/upload/{upload_id}/part/0"}
376:@app.put("/api/upload/{upload_id}/part/{index}")
410:@app.post("/api/upload/{upload_id}/complete")
412:    """Finalize: verify part count, move into place, return {path, video_id} for
/api/process."""
431:            "next": "POST /api/process with this path"}
434:@app.delete("/api/upload/{upload_id}")
447:@app.get("/api/highlights/{video_id}/{filename}")
449:    """Stream a compiled highlight reel ? `filename` is the bare name given to /api/compile
466:                            detail=f"No compiled file named '{name}'. Compile first via
/api/compile.")
471:@app.post("/api/query")
482:@app.post("/api/compile")
680:    # /api/compile ? reads the same value instead of a write-only runtime hack.
```

24. user (2026-06-14T08:24:53.527Z)

```
backend\pipeline\detectors\audio_features.py:5:per-window features the learned scorer can
weight. Mirrors `fusion_features.py` discipline.
backend\pipeline\detectors\fusion_features.py:194:PERSON_FEATURE_NAMES = [
backend\pipeline\detectors\fusion_features.py:200:def compute_person_features(per_frame:
Dict[int, List[Tuple[int, BBox]]], shuttle_points,
backend\pipeline\detectors\fusion_features.py:206:    shuttle trajectory. Pure; returns exactly
`PERSON_FEATURE_NAMES`, all floats."""
backend\pipeline\detectors\person_detector.py:268:
``fusion_features.shuttle_player_colocation`` / ``two_sided_motion`` rely on.
backend\pipeline\segmenters\fusion_hybrid.py:11:  (`fusion_features`), persisting them into
``candidate_segments.stats`` for the learned
backend\pipeline\segmenters\fusion_hybrid.py:16:- ``_select_for_handoff`` ? optional served Gate
A/B: when ``indexing.fusion.min_crossings``
backend\pipeline\segmenters\fusion_hybrid.py:22:Defaults reproduce the substrate exactly:
net_crossings/person features are persisted but
backend\pipeline\segmenters\fusion_hybrid.py:36:from backend.pipeline.detectors import
fusion_features as ff
backend\pipeline\segmenters\fusion_hybrid.py:84:
min_crossings=0, estimate=self._axis_estimate(points))
backend\pipeline\segmenters\fusion_hybrid.py:85:        stats["net_crossings"] =
float(gated[0].crossings) if gated else 0.0
backend\pipeline\segmenters\fusion_hybrid.py:124:            return {k: 0.0 for k in
ff.PERSON_FEATURE_NAMES}
backend\pipeline\segmenters\fusion_hybrid.py:130:        return
```

```
ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
backend\pipeline\segmenters\fusion_hybrid.py:140:        min_crossings =
int(fcfg.get("min_crossings", 0))
backend\pipeline\segmenters\fusion_hybrid.py:146:            if min_crossings > 0 and
st.get("net_crossings", 0.0) < min_crossings:
backend\pipeline\detectors\rally_gate.py:69:def count_net_crossings(window_points, axis: str,
net: float, deadband: float) -> int:
backend\pipeline\detectors\rally_gate.py:92:                min_crossings: int = 2,
deadband_frac: float = 0.06,
backend\pipeline\detectors\rally_gate.py:95:    keep decision (crossings >= min_crossings).
Deadband is a fraction of the play
backend\pipeline\detectors\rally_gate.py:106:        c = count_net_crossings(wp, axis, net,
deadband)
backend\pipeline\detectors\rally_gate.py:109:                                kept=c >=
min_crossings))
backend\pipeline\detectors\rally_gate.py:114:               min_crossings: int = 2,
deadband_frac: float = 0.06) -> List[Interval]:
backend\pipeline\detectors\rally_gate.py:117:            gate_windows(points, windows, fps,
min_crossings, deadband_frac) if g.kept]
backend\pipeline\segmenters\trajectory_hybrid.py:87:        rows; ``FusionSegmenter`` overrides
this to add ``net_crossings`` (+ person-cue
backend\pipeline\segmenters\trajectory_hybrid.py:201:        # segmenter adds net_crossings +
person-cue features. Computed once, reused for both
backend\pipeline\segmenters\trajectory_hybrid.py:212:                candidate_ids[i] =
self.db.add_candidate_segment(
backend\pipeline\segmenters\yolo_hybrid.py:260:               candidate_ids[i] =
self.db.add_candidate_segment(
```

25. assistant (2026-06-14T08:24:58.860Z)

There's a `fusion_hybrid.py` segmenter that persists the cue features into production candidate stats. This is significant for the gap analysis. Let me read it and check the correction loop in main.py for how human labels become per-user labels.

26. user (2026-06-14T08:25:00.126Z)

```
1       """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3       Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4       TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5       do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6       shuttle-seeded window via two overridable seams the engine exposes:
7
8       - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9         GPU-free, computed from the trajectory we already have); and, only when the person cue
10        is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11        (`fusion_features`), persisting them into ``candidate_segments.stats`` for the learned
12        fusion (P3). The person path lazily builds ONE `PersonDetector` and only then loads
the
13        torchvision/supervision model ? so the default (person-OFF) path never loads a
detector
14        model nor touches the GPU. (torch/cv2 themselves are imported by the registry
regardless,
15        via the pre-existing yolo_hybrid ? person_detector chain; the lazy part is the model.)
16      - ``_select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
17        (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
18        dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19        **Persisted candidates remain the full superset** regardless, so the offline
experiment
20        harness can re-score any variant.
21
22      Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
```

```
    nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
    and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
    segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
    experiment harness (EXPERIMENT_HARNESS.md), never silently.
    """
    from typing import Any, Dict, List, Optional, Tuple

    from backend.pipeline.segmenters.base import segmenter_registry
    from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
    from backend.pipeline.detectors.base import DetectorRunner
    from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
    from backend.pipeline.detectors import rally_gate
    from backend.pipeline.detectors import fusion_features as ff


    class FusionSegmenter(TrajectoryHybridSegmenter):
        """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
B)."""

        family = "fusion"
        display_label = "Fusion"

        def __init__(self, db: Any, config: Any):
            super().__init__(db, config)
            # Built lazily only if the person cue is enabled (no detector model loaded
otherwise).
            self._person: Optional[Any] = None
            # Cache the play-axis/net estimate per trajectory so rally_gate doesn't re-
estimate it
            # over the whole trajectory once per candidate window (it depends only on
`points`).
            self._axis_cache_key: Optional[int] = None
            self._axis_cache: Optional[tuple] = None

        @property
        def name(self) -> str:
            return "fusion_hybrid"

        @property
        def description(self) -> str:
            return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
rallies, "
                    "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
and the "
                    "AI provider sets semantic boundaries.")

        def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
            substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
            if substrate == "tracknet":
                cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
                return TrackNetRunner(cfg), {
                    "weights_path": cfg.weights_path,
                    "queue_length": cfg.queue_length,
                    "fusion_substrate": "tracknet",
                }
            wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
            return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"}

        def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
```

```
                                    frame_width: float, video_path: str,
                                    idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
            stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
video_path, idx_cfg)
            # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
            # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
            # over the whole trajectory by rally_gate (camera-agnostic); reused across
windows.
            gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
                                            min_crossings=0,
estimate=self._axis_estimate(points))
            stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0

            fcfg = idx_cfg.get("fusion", {})
            if fcfg.get("cue_flags", {}).get("person", False):
                stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
            return stats

        def _axis_estimate(self, points: List[Any]) -> tuple:
            """Cache rally_gate's (axis, net, span) play-axis estimate per trajectory so
it's
            computed once per video, not once per candidate window (it depends only on
points)."""
            key = id(points)
            if self._axis_cache_key != key or self._axis_cache is None:
                self._axis_cache_key = key
                self._axis_cache = rally_gate.estimate_play_axis(points)
            est = self._axis_cache
            assert est is not None
            return est

        def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
                             frame_width: float, video_path: str,
                             fcfg: Dict[str, Any]) -> Dict[str, float]:
            """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
            frame range and compute the scale/translation-invariant person features. Lazily
builds
            ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
            # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
            from backend.pipeline.detectors.person_detector import PersonDetector
            if self._person is None:
                self._person = PersonDetector(self.config)

            frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
            # round (not truncate): int() would map e.g. 1.999s*30 to frame 59 not 60 ? a
            # window-edge off-by-one that drops the boundary frame from the shuttle
alignment.
            start_f, end_f = round(cw["start"] * fps), round(cw["end"] * fps)
            det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
            fw = det["frame_width"] or frame_width
            fh = det["frame_height"]
            # If we can't trust the frame dims, every normalised feature would silently
collapse
            # to ~0 ? return an explicit all-zero person-feature row instead of a half-baked
one.
            if fw <= 0 or fh <= 0:
                return {k: 0.0 for k in ff.PERSON_FEATURE_NAMES}

            poly = fcfg.get("court_polygon")
```

```
127             polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
128             net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
129             shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
130             return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
131                                               court_polygon=polygon, net=net)
132
133         def _select_for_handoff(self, windows: List[Dict[str, Any]],
134                                 window_stats: List[Dict[str, Any]],
135                                 idx_cfg: Dict[str, Any]) -> List[int]:
136             """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
137             (0) keep every window ? identical to the substrate. Persisted candidates are
138             untouched (full superset)."""
139             fcfg = idx_cfg.get("fusion", {})
140             min_crossings = int(fcfg.get("min_crossings", 0))
141             min_players = int(fcfg.get("min_players", 0))
142             person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
143
144             keep: List[int] = []
145             for i, st in enumerate(window_stats):
146                 if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
147                     continue
148                 if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
149                     continue
150                 keep.append(i)
151             return keep
152
153
154     segmenter_registry.register(FusionSegmenter)
155
```

27. user (2026-06-14T08:25:01.450Z)

```
245-                )
246-
247-        # Persist raw candidates for the fine-tuning dataset. Map window index ->
candidate_id
248:        # so confirmed AI segments can be linked back in Phase 4.
249-        candidate_ids = [None] * len(all_candidate_windows)
250-        if store_candidates:
251-            for i, w in enumerate(all_candidate_windows):
--
296-                try:
297-                    segment["start_time"] = float(segment["start_time"]) + w_start
298-                    segment["end_time"] = float(segment["end_time"]) + w_start
299:                    # Stamp the source candidate so Phase 4 can link
prediction->confirmation.
300-                    segment["_candidate_id"] = candidate_ids[wi]
301-                    valid_segments.append(segment)
302-                except (ValueError, TypeError, KeyError) as e:
--
320-        except OSError:
321-            pass
322-
323:        logger.info(f"Phase 3 completed. AI confirmed {len(gemini_segments)} semantic play
segments.")
324-
325-        # Phase 4: DB Population
326-        segments_data = []
--
348-                segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
349-                if segment_id:
350-                    # Link back to the candidate that produced this segment. A candidate
```

```
351:                    # may yield multiple AI segments (one-to-many); confirmed_segment_id
keeps
352:                    # the last one ? the link table records "this candidate was confirmed".
353-                    candidate_id = segment.get("_candidate_id")
354-                    if candidate_id is not None:
355:                        self.db.link_candidate_to_segment(candidate_id, segment_id)
356-                    segments_data.append({
357-                        "id": segment_id,
358-                        "start_time": start,
```

28. assistant (2026-06-14T08:25:04.853Z)

Now I have a clear picture of the candidate?confirmation link. Let me check how human corrections set `human_label` (the actual per-user label source), and check the NEXT_STEPS / fusion plan docs to see if personalization is mentioned anywhere as a design intent. Let me also confirm there is no per-user model store or per-user eval reporting.

29. user (2026-06-14T08:25:07.361Z)

```
tests\test_api_endpoints.py:96:def test_process_rejects_null_byte_path(env):
tests\test_api_endpoints.py:111:def test_compile_rejects_traversal_output_filename(env):
backend\infrastructure\database.py:110:                        human_label TEXT,
backend\infrastructure\database.py:299:            query += " AND human_label IS NULL"
tests\test_async_jobs.py:120:def test_submit_rejects_null_byte_path(env):
backend\config\models.py:214:    # contain upload_dir or /api/process will reject the uploaded
paths.
tests\test_database.py:59:    "confirmed_segment_id", "human_label", "notes", "created_at",
tests\test_database.py:149:    # only_unlabeled returns it while human_label is NULL
tests\test_database.py:152:        conn.execute("UPDATE candidate_segments SET human_label = ?
WHERE id = ?",
backend\eval\gemini_refine.py:1:"""Refine WASB rally candidates with Gemini (verify boundaries +
reject false fires).
tests\test_eval_annotations.py:77:def test_load_rejects_start_after_end(tmp_path):
tests\test_eval_annotations.py:83:def test_load_rejects_short_row(tmp_path):
backend\main.py:41:# rejected by browsers per the fetch spec and is an unsafe default to carry
into the
backend\main.py:51:# Serves static frontend files directly (now under api/ per approved
structure)
backend\pipeline\validators\blur.py:75:                message=f"Video quality rejected: Focus
check failed. Average sharpness score of {round(avg_variance, 1)} is below the minimum required
threshold of {min_blur_threshold}.",
backend\pipeline\validators\relevance.py:101:                message=f"Video rejected: Relevance
check failed. The scene does not contain a recognizable {sport_name} court (court area found:
{round(avg_green_ratio * 100, 2)}% vs min required {round(min_court_ratio * 100, 2)}%).",
backend\pipeline\segmenters\gemini.py:348:        rejected = []
backend\pipeline\segmenters\gemini.py:355:                rejected.append((label, f"start_time
({c['start']:.2f}s) is beyond video duration ({video_duration:.1f}s)"))
backend\pipeline\segmenters\gemini.py:367:                rejected.append((label, f"duration
({dur:.2f}s) is below minimum ({min_segment_duration}s)"))
backend\pipeline\segmenters\gemini.py:393:                    rejected.append((dropped_label,
f"overlaps with {kept_label} ? kept the longer segment"))
backend\pipeline\segmenters\gemini.py:401:        if rejected:
backend\pipeline\segmenters\gemini.py:402:            reject_lines = [f"  ? {label}: {reason}"
for label, reason in rejected]
backend\pipeline\segmenters\gemini.py:403:            logger.info(f"{len(rejected)} play
segments were REJECTED:\n" + "\n".join(reject_lines))
tests\test_path_safety.py:12:def test_safe_output_filename_rejects(bad):
tests\test_path_safety.py:22:def test_validate_input_path_rejects_null_byte():
tests\test_provider_gemini.py:55:def
test_gemini_provider_rejects_oversize_before_upload(monkeypatch, caplog):
tests\test_reliability_hardening.py:20:def test_chunk_overlap_ge_duration_rejected():
tests\test_tracknet_runner.py:149:def test_run_predict_rejects_bad_extension(tmp_path):
tests\test_upload_download.py:46:def test_init_rejects_bad_extension(env):
tests\test_upload_download.py:53:def test_init_rejects_traversal_filename(env):
tests\test_upload_download.py:59:def test_init_rejects_oversize_declared_total(env):
```

```
tests\test_upload_download.py:168:def test_download_traversal_filename_rejected(env):
```

30. user (2026-06-14T08:25:08.684Z)

```
Found 12 files
docs\CODE_MAP.md
docs\STORAGE_SHARING_MODEL.md
docs\VIDEO_LOCALITY_MODEL.md
docs\PLATFORM_ARCHITECTURE.md
docs\FIELD_VISIT_PLAYBOOK.md
docs\UI_ALPHA_EXECUTION_PLAN.md
docs\UI_PROPOSAL.md
docs\HOSTING_PLAN.md
docs\OWNED_MODEL_IMPLEMENTATION_PLAN.md
docs\archives\research\OWNED_MODEL_TRAINING_STUDY.md
docs\archives\checkpoints\2026-06-strategy-foundation\README.md
docs\archives\checkpoints\2026-06-scaffolding-complete\README.md
```

31. assistant (2026-06-14T08:25:15.517Z)

```
There's no `human_label` setter in production code ? only a test sets it directly via SQL. The
correction loop is therefore NOT wired as a write path yet. Let me confirm how the owned-model
plan frames per-user training and check whether personalization is explicitly part of the
roadmap, to fairly assess the owner's "designed for this" claim.
```

32. user (2026-06-14T08:25:18.470Z)

```
29:- **Exclude minors; per-user training data needs a separate explicit opt-in; split data by
match.**
31:  training" rule is enforceable at *our fine-tuning data* layer, not the base's pretraining ?
an **explicit owner
46:- **START WITH THE TAL HEAD, NOT THE VLM** (the decisive call): fine-tuned VLMs **miss strict
temporal boundaries**
72:- **Model/framework: UNCHANGED.** One shared TAL head + one Qwen3-VL base, fine-tuned across
sports (sport = an input
144:- **Gen-2 (VLM):** ms-swift fine-tune of **Qwen3-VL** (SFT cold-start ? GRPO/RFT with a
temporal-IoU-vs-golden reward,
```

33. user (2026-06-14T08:25:19.966Z)

```
65:     unlabeled footage** ? agreed/A-only/B-only), `record_experiment` (leaderboard JSON), and
the DB
95:   2070S ? still lightweight, NOT the heavyweight Gen-2 stack) inside the existing harness;
per-video threshold
```

34. assistant (2026-06-14T08:25:25.155Z)

```
I have enough to assess. The owned-model plan mentions "per-user training data needs a separate
explicit opt-in" and "split data by match" ? but framed as ONE shared model across sports/users,
not per-user models. Let me check the `paired_delta_ci`/leaderboard for any per-corpus keying
and confirm the LOVO is the only cross-validation unit (no per-user split concept). Let me
verify the leaderboard record shape and the manifest concept one more time.
```

35. user (2026-06-14T08:25:25.526Z)

```
98      def run(specs: List[Dict], iou: float = 0.5, threshold: float = 0.5,
99              model_out: Optional[str] = None) -> dict:
100         videos = [load_video(s, iou) for s in specs]
101         print(f"=== Distilled local rally classifier vs Gemini silver-GT "
102               f"({len(videos)} videos, IoU>={iou}, prob>={threshold}) ===")
103         for v in videos:
104             ceil = score(v["intervals"], v["gts"], iou).recall
105             print(f"[{v['name']}] {len(v['intervals'])} candidates ({sum(v['y'])} pos) | "
106                   f"proposal recall ceiling {ceil:.3f} | {len(v['gts'])} Gemini rallies")
```

```
107
108         result: dict = {"videos": [v["name"] for v in videos]}
109         if len(videos) >= 2:
110             print("\n--- HONEST leave-one-video-out: learned vs threshold baseline (held-
out) ---")
111             clf, base = [], []
112             for i, held in enumerate(videos):
113                 others = [v for j, v in enumerate(videos) if j != i]
114                 X = [row for v in others for row in v["X"]]
115                 y = [t for v in others for t in v["y"]]
116                 model = LogisticRegression().fit(X, y, FEATURE_NAMES)
117                 preds = predict_rallies(model, held["X"], held["intervals"], threshold)
118                 sc = score(preds, held["gts"], iou)
119                 bsc = score(held["baseline"], held["gts"], iou)
120                 clf.append(sc)
121                 base.append(bsc)
122                 print(f"  held-out {held['name']}: LEARNED P={sc.precision:.3f}
R={sc.recall:.3f} F1={sc.f1:.3f}"
123                       f"  |  baseline P={bsc.precision:.3f} R={bsc.recall:.3f}
F1={bsc.f1:.3f}")
124             def mean(xs, a):
125                 return sum(getattr(s, a) for s in xs) / len(xs)
126             print(f"  MEAN held-out: LEARNED P={mean(clf,'precision'):.3f}
R={mean(clf,'recall'):.3f} "
127                   f"F1={mean(clf,'f1'):.3f}  |  baseline P={mean(base,'precision'):.3f} "
128                   f"R={mean(base,'recall'):.3f} F1={mean(base,'f1'):.3f}")
129             result["mean_learned_f1"] = mean(clf, "f1")
130             result["mean_baseline_f1"] = mean(base, "f1")
131
132         # Final model on ALL videos (for production / running on a fresh held-out video).
133         Xall = [row for v in videos for row in v["X"]]
134         yall = [t for v in videos for t in v["y"]]
135         final = LogisticRegression().fit(Xall, yall, FEATURE_NAMES)
136         assert final.w is not None  # set by fit()
137         weights = dict(zip(FEATURE_NAMES, [round(float(w), 3) for w in final.w]))
138         print(f"\nfinal model standardized weights (sign/size = influence): {weights}")
139         if model_out:
140             final.save(model_out)
141             print(f"final model (all videos) -> {model_out}")
142             result["model_out"] = model_out
143         return result
144
145
146     def main() -> None:
147         ap = argparse.ArgumentParser(description="Distill Gemini labels into a local rally
classifier.")
148         ap.add_argument("--manifest", required=True,
149                         help="JSON: [{name, trajectory, fps, frame_width, labels}, ...]
(labels = Gemini CSV)")
150         ap.add_argument("--iou", type=float, default=0.5)
151         ap.add_argument("--threshold", type=float, default=0.5, help="classifier probability
cutoff")
152         ap.add_argument("--model-out", default="output/local_rally_model.json")
153         args = ap.parse_args()
154         with open(args.manifest, encoding="utf-8") as f:
155             specs = json.load(f)
156         run(specs, iou=args.iou, threshold=args.threshold, model_out=args.model_out)
157
```

36. assistant (2026-06-14T08:25:29.645Z)

I now have a complete picture. The `run()` already produces a "final model on ALL videos for running on a fresh held-out video" ? which is exactly the per-corpus model-fitting primitive. Let me do one final check: confirm there is no per-user keying anywhere (the corpus/manifest is just a list of videos with no user dimension) and no per-user model versioning store.

37. user (2026-06-14T08:25:31.658Z)

Found 1 file
backend\pipeline\detectors\wasb_infer.py

38. user (2026-06-14T08:25:32.885Z)

No matches found

39. assistant (2026-06-14T08:25:38.252Z)

Confirmed: the learned `LogisticRegression` classifier is NOT loaded anywhere in the production `backend/pipeline` path. The `classifier_model` variant exists only in the eval harness ? the served pipeline uses gates/thresholds, not the learned model. This is a critical gap for the personalization pivot (the per-user model has no path to production serving).

I have everything needed. Let me compile the structured findings.

40. user (2026-06-14T08:26:56.499Z)

Structured output provided successfully

41. assistant (2026-06-14T08:27:11.025Z)

I've completed the analysis. Summary of what I verified against the code:

The owner's "designed for this" claim is PARTIALLY TRUE ? genuinely strong at the decision layer, overstated for the end-to-end loop.

What's already built and reusable for per-user optimization:
- `compare()`/LOVO (`backend/eval/experiment.py:296-448`) is corpus-relative ? pass one user's videos and you get a per-user A/B with no new code.
- `Variant` cue flags + Gate A/B thresholds (`experiment.py:76-153`) ARE the per-user "person cue off for this user" switch; promote-only-on-lift via `eval_stats.py:90-109` is per-user cue gating.
- The replayable per-video feature JSON substrate (`backend/eval/fusion_golden.py`) is exactly "detect once on GPU, re-decide forever on CPU as the corpus grows."
- Honest small-N stats (C1?C4) were built for the n?6 regime ? which *is* the per-user regime.
- `FusionSegmenter` already persists cue features into production `candidate_segments.stats` with served gate knobs (`fusion_hybrid.py:76-151`); `distill_local.run()` already fits a per-corpus model and saves it as a pinnable JSON.

The three load-bearing gaps the pivot must fund:
1. **No serving path for a learned per-user model** ? `LogisticRegression.load` is referenced ONLY in `backend/eval/*`; the served `FusionSegmenter` uses gates, not the learned model (grep of `backend/pipeline` = zero loads). The per-user model is an artifact production can't consume.
2. **The correction-loop ? per-user-labels write path doesn't exist** ? `human_label` is never SET in production (only a test writes it via raw SQL); no correction/reject API in `main.py`. The schema column + `link_candidate_to_segment` are stubs.
3. **No user/tenant dimension** ? no per-user model store, versioning, eval partition, or continual/scheduled re-fit. The harness was designed single-corpus (`EXPERIMENT_HARNESS.md §9`).

Key asymmetry: the harness supports per-user *tuning* (gate thresholds, cue ON/OFF ? serveable today) far better than per-user *learning* (a model fit to the user's footage ? not serveable). Also flagged: the deliberate court/camera-invariance of the features (`fusion_features.py:1-9`) partly engineers away the pivot's "wins won't transfer to other users" specialization upside. Full pros/cons/gaps returned via StructuredOutput.